

Universidad Nacional Pedro Ruiz Gallo
Escuela Profesional de Ingeniería de Sistemas — MATG1001

LogiLearn
Leyes lógicas y tablas de verdad

Inicio Aprender Tablas de verdad Leyes lógicas Ejercicios Progreso

LÓGICA MATEMÁTICA DE FORMA VISUAL E INTERACTIVA

Aprende Lógica Simbólica de forma visual e interactiva y desarrolla el razonamiento lógico con ejercicios, simulaciones y seguimiento de tu progreso.

Una plataforma colaborativa gratuita, de código abierto, que permite a los usuarios crear, modificar y compartir contenidos de manera conjunta, Fortaleciendo sus competencias en lógica proposicional, inferencias lógicas y teoría de conjuntos mediante recursos interactivos, ejercicios de aplicación y un seguimiento sistemático de tu progreso.

Empezar a aprender Explorar LogiLearn

4 Módulos interactivos	12 Leyes lógicas	80 Ejercicios	100% Silabo cubierto
---------------------------	---------------------	------------------	-------------------------

Manual Técnico

LogiLearn — Arquitectura, motores y sprints por equipo

Plataforma LogiLearn — Lógica Simbólica Global
Docente: Dr. Mardo Victor Gonzales Herrera — Septiembre 2026

Índice

1. Capítulo 1 — Sinergia: motor proposicional	5
2. Introducción	1
3. Marco Teórico	1
3.1. Lógica Proposicional y Conectivos Lógicos	1
3.2. Clasificación Semántica de Proposiciones	2
3.3. Árbol de Sintaxis Abstracta (AST) y Precedencia	2
3.4. Gramática BNF del Motor	3
4. Arquitectura del Sistema	3
4.1. Stack Tecnológico	3
4.2. Estructura de Archivos del Módulo Sinergia	3
5. Explicación Microscópica del Código	4
5.1. Motor Lógico: <code>src/lib/truth-table/evaluator.ts</code>	4
5.1.1. Tipos e Interfaces TypeScript (líneas 1–32)	4
5.1.2. Clase de Error Personalizada (líneas 34–39)	4
5.1.3. Mapa de Símbolos y Normalización (líneas 41–64)	4
5.1.4. Tokenizador con Autómata Finito Determinista (líneas 66–85)	4
5.1.5. Clase Parser: Análisis Descendente Recursivo (líneas 87–175)	5
5.1.6. Función <code>generarFilas</code> con Aritmética de Bits (líneas 269–298)	5
5.2. Vista Principal: <code>src/pages/tablas/index.vue</code>	5
6. Manual de Usuario — Guía Paso a Paso	5
7. Pruebas y Validación	6
8. Conclusiones	6
9. Capítulo 2 — Hijos de Linus: motor de inferencia	12
10. Introducción y Propuesta del Proyecto (Sprint 1)	3
11. Arquitectura del Software y Flujo de Trabajo	4
11.1. Estructura de Directorios del Módulo	4
11.2. Flujo de Git por Ramas	6
12. Capa Lógica: El Motor de Inferencia y Demostración (Sprint 2)	7
12.1. Código Fuente del Motor Lógico	7
13. Capa Visual: Interfaz Reactiva y Didáctica (Sprint 3 y 3.5)	9
13.1. Reactividad en Vue 3 y Composition API	9
13.2. Trazabilidad Pedagógica y Herramientas Académicas	9
13.3. Visualización Interactiva del Árbol de Sintaxis Abstracta (AST)	9
14. Pruebas de Calidad del Código (Sprint 4)	11
15. Participación y Aportes del Equipo	13
16. Trabajo en Equipo	13
17. Conclusiones	13

18.Capítulo 3 — Modus Innova: predicados y leyes	13
19.CAPÍTULO I: RESUMEN EJECUTIVO Y ABSTRACT	13
19.1. 1.1 Resumen Ejecutivo (Español)	13
19.2. 1.2 Abstract (English)	16
19.3. 1.3 Declaración Formal de Autoría	17
20.CAPÍTULO II: FUNDAMENTACIÓN TEÓRICA Y MATEMÁTICA	17
20.1. 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)	17
20.2. 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)	18
20.3. 2.3 Leyes de De Morgan en Cuantificadores	19
20.4. 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)	20
21.CAPÍTULO III: BITÁCORA EVOLUTIVA MINUCIOSA (4 SEMANAS + FASE POSTERIOR)	21
21.1. 3.1 Semana 1: Concepción y Pivotaje	21
21.2. 3.2 Semana 2: Motores TS y el Dilema de Leyes	22
21.3. 3.3 Semana 3: Integración en Vue 3 y PR en GitHub	23
21.4. 3.4 Semana 4: Pruebas QA y Bug de Implicación	24
21.5. 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor	25
22.CAPÍTULO V: EXPLICACIÓN MICROSCÓPICA LÍNEA POR LÍNEA DEL CÓDIGO FUENTE COMPLETO	26
23.CAPÍTULO VI: MATRIZ EXHAUSTIVA DE 20 CASOS DE PRUEBA DIDÁCTICOS (QA & TESTING)	30
24.CAPÍTULO VII: MATRIZ DE INTEGRANTES (7 MIEMBROS), CONCLUSIONES Y FIRMA	30
24.1. 7.2 Conclusiones y Firma Oficial	31
25.Capítulo 4 — Linus: conjuntos y Venn	31
Resumen / Abstract	2
26.Introducción	3
26.1. Contexto y problema	3
26.2. Motivación del equipo	3
26.3. Objetivo general	3
26.4. Objetivos específicos	3
26.5. Alcance del proyecto	4
26.6. Estructura del informe	4
27.Marco Teórico	5
27.1. Teoría de conjuntos - conceptos fundamentales	5
27.2. Operaciones básicas sobre conjuntos	5
27.2.1. Unión ($A \cup B$)	5
27.2.2. Intersección ($A \cap B$)	5
27.2.3. Diferencia ($A \setminus B$)	5
27.2.4. Complemento (A')	5
27.3. Conjunto potencia	6
27.4. Diagramas de Venn	6
27.5. Tecnologías de la plataforma web moderna	6

28.Estado del Arte	7
28.1. Herramientas similares existentes	7
28.2. Comparativa de tecnologías frontend	7
28.3. Justificación de la selección tecnológica	7
29.Metodología	9
29.1. Proceso de desarrollo ágil	9
29.2. Git Flow y control de versiones	9
29.3. Arquitectura del software	10
29.4. Estructura de archivos del proyecto	10
30.Implementación del Motor Lógico	12
30.1. Archivo: src/lib/sets/operations.ts	12
30.2. Función: union<T>(a: Set<T>, b: Set<T>): Set<T>	12
30.3. Función: interseccion<T>(a: Set<T>, b: Set<T>): Set<T>	13
30.4. Función: diferencia<T>(a: Set<T>, b: Set<T>): Set<T>	13
30.5. Función: complemento<T>(universo: Set<T>, a: Set<T>): Set<T>	14
30.6. Función: potencia<T>(a: Set<T>): Set<Set<T>>	14
30.7. Análisis de complejidad consolidado	15
31.Implementación de la Interfaz de Usuario	16
31.1. Estructura del Single File Component (SFC)	16
31.2. Variables reactivas	16
31.3. Función parseSet()	17
31.4. Propiedad computada: resultado	17
31.5. Diagrama SVG de Venn	18
31.6. Estilos y responsividad	18
32.Cambios y Mejoras Realizados Durante el Desarrollo	19
32.1. Proceso de refinamiento iterativo	19
32.2. Cambio 1 - Eliminación de texto redundante en resultados	19
32.3. Cambio 2 - Cuadro estadístico separado para el conteo 2^n	19
32.4. Cambio 3 - Corrección completa de la operación complemento	20
32.5. Cambio 4 - Sub-selector de conjunto potencia	20
32.6. Cambio 5 - Corrección del diagrama SVG en diferencias y complementos	21
32.7. Cambio 6 - Normalización de textos de la interfaz	21
32.8. Impacto consolidado de los seis cambios	22
33.Pruebas y Validación	23
33.1. Estrategia de pruebas	23
33.2. Configuración de Vitest	23
33.3. Casos de prueba completos - Motor lógico	23
33.3.1. Suite: union()	23
33.3.2. Suite: interseccion()	24
33.3.3. Suite: diferencia()	24
33.3.4. Suite: complemento()	24
33.3.5. Suite: potencia()	24
33.4. Resumen de cobertura de pruebas	24
33.5. Pipeline de CI/CD - GitHub Actions	24
33.6. Pruebas manuales de interfaz de usuario	25

34.Registro de Evolución Semanal del Proyecto	27
34.1. Visión general del desarrollo	27
34.2. Semana 1 - Infraestructura (31 julio)	27
34.3. Semana 2 - Fundamentos colaborativos (1-6 agosto)	27
34.4. Semana 3 - Propuestas individuales (7 agosto)	27
34.5. Semana 4 - Implementación del motor lógico (8-17 agosto)	28
34.6. Semana 5 - Interfaz, CI/CD y refinamiento (19-30 agosto)	28
34.7. Estadísticas del proyecto	29
35.Resultados y Discusión	30
35.1. Funcionalidades implementadas vs planificadas	30
35.2. Rendimiento medido	30
35.3. Comparación con el objetivo inicial	30
35.4. Limitaciones conocidas y trabajo futuro	30
35.5. Trabajo futuro recomendado	30
36.Conclusiones	33
36.1. Logro del objetivo principal	33
36.2. Calidad del motor lógico	33
36.3. Efectividad de Vue 3 para aplicaciones educativas	33
36.4. Valor del desarrollo ágil iterativo	33
36.5. Importancia de la documentación continua	33
36.6. Aprendizajes del equipo	33
36.7. Impacto educativo potencial	33
Referencias Bibliográficas	35
A. Apéndice A - Código Completo del Motor Lógico	36
Apéndice A - Código completo operations.ts	36
B. Apéndice B - Código de Pruebas Unitarias	39
Apéndice B - Código pruebas unitarias	39
C. Apéndice C - Configuración del Proyecto	42
Apéndice C - Configuración del proyecto	42
C.1. package.json	42
C.2. tsconfig.json	42
C.3. vite.config.ts	43
D. Apéndice D - Glosario de Términos	44
Apéndice D - Glosario de términos	44
E. Estructura del repositorio (Árbol Sinergia)	45

1. Capítulo 1 — Sinergia: motor proposicional

% =====
PORTADA INSTITUCIONAL % =====

UNIVERSIDAD NACIONAL PEDRO RUIZ GALLO

Escuela Profesional de Ingeniería de Sistemas

Lógica Simbólica · MATG1001 · Semestre 2026-I · II Ciclo

**INFORME TÉCNICO DEL MÓDULO DE TABLAS DE
VERDAD:
DISEÑO, IMPLEMENTACIÓN Y DOCUMENTACIÓN
DEL MOTOR LÓGICO PROPOSICIONAL**

Proyecto LogiLearn – Lógica Simbólica Global

Equipo Sinergia

Sublíder: Alexa Benavides Irigoin

Integrantes: Aldair Querebalu · Luis Atencio · Miguel Velarde · Jesús Núñez

Docente: Dr. Mardo Victor Gonzales Herrera

Líder de Proyecto: Enrique Díaz Cayaca (EnriDiazCayaca)

Lambayeque, Perú · Septiembre 2026

Repositorio: github.com/EnriDiazCayaca/logica_simbolica_global

% =====
RESUMEN (ESTÁNDAR SCIELO) % =====

RESUMEN

Objetivo: Documentar de forma exhaustiva el diseño, la arquitectura, el código fuente y el proceso de desarrollo del módulo de Tablas de Verdad implementado por el Equipo Sinergia en el marco del proyecto colaborativo LogiLearn, correspondiente al curso Lógica Simbólica (MATG1001) de la Universidad Nacional Pedro Ruiz Gallo.

Metodología: Se adoptó una metodología ágil de cuatro sprints (Propuesta y Diseño, Motor Lógico, Interfaz Visual, Pruebas y Despliegue), ejecutados del 3 al 30 de agosto de 2026. El stack tecnológico % empleó Vue 3 con Composition API, TypeScript en modo estricto, Vite 6, Tailwind CSS v4 y Vitest.

Resultados: Se desarrolló un motor lógico completo capaz de analizar, tokenizar, parsear y evaluar fórmulas de lógica proposicional con soporte para cinco conectivos (negación, conjunción, disyunción, implicación y bicondicional) en múltiples notaciones. El módulo genera tablas de verdad con sub-expresiones intermedias y clasifica automáticamente la proposición como tautología, contradicción o contingencia. Se implementaron 25 pruebas unitarias con cobertura del 100 % de las rutas de evaluación.

Palabras clave: lógica proposicional, tablas de verdad, análisis sintáctico, árbol de sintaxis abstracta, Vue 3, TypeScript, tautología, contradicción, contingencia.

% =====
1. INTRODUCCIÓN % =====

2. Introducción

La lógica proposicional constituye la base formal del razonamiento matemático y computacional. Las tablas de verdad, herramienta fundamental de este campo, permiten determinar el valor de verdad de una fórmula compleja para cada posible combinación de valores de sus variables proposicionales. A pesar de su importancia pedagógica, no existen plataformas web de código abierto, en español y orientadas a estudiantes universitarios de habla hispana, que implementen este proceso de forma interactiva con trazabilidad completa de sub-expresiones.

El presente informe documenta el trabajo del **Equipo Sinergia**, responsable del módulo *Tablas de Verdad* dentro del proyecto colaborativo **LogiLearn**, desarrollado por el aula del curso Lógica Simbólica (MATG1001, Semestre 2026-I) de la Universidad Nacional Pedro Ruiz Gallo. El módulo implementa un motor lógico de propósito general, capaz de procesar entradas en múltiples notaciones, construir un árbol de sintaxis abstracta (AST), evaluar la fórmula y clasificarla semánticamente.

La documentación sigue el formato de artículo técnico compatible con SciELO y está organizada en las siguientes secciones: (2) Marco Teórico, (3) Arquitectura del Sistema, (4) Explicación Microscópica del Código, (5) Manual de Usuario, (6) Evolución por Sprints, (7) Pruebas y Validación, y (8) Conclusiones.

% =====
2. MARCO TEÓRICO % =====

3. Marco Teórico

3.1. Lógica Proposicional y Conectivos Lógicos

La lógica proposicional es un sistema formal en el que las proposiciones son entidades que pueden tomar un valor de verdad: verdadero (V) o falso (F). Los conectivos lógicos permi-

ten combinar proposiciones atómicas para formar fórmulas compuestas. Los cinco conectivos implementados en el motor son:

- **Negación** ($\neg$ / **NOT**): operador unario que invierte el valor de verdad. $\neg V = F$; $\neg F = V$.
- **Conjunción** ($\wedge$ / **AND**): verdadera si y solo si ambos operandos son verdaderos.
- **Disyunción** ($\vee$ / **OR**): verdadera si al menos uno de los operandos es verdadero.
- **Implicación** ($\rightarrow$ / **IMPLIES**): falsa únicamente cuando el antecedente es V y el consecuente es F. Equivale a $\neg P \vee Q$.
- **Bicondicional** ($\leftrightarrow$ / **IFF**): verdadera cuando ambos operandos tienen el mismo valor de verdad.

Tabla 1: Tablas de verdad de los cinco conectivos lógicos

$P \ \& \ Q \ \& \ \neg P \ \& \ P \wedge Q \ \& \ P \vee Q \ \& \ P \rightarrow Q \ \& \ P \leftrightarrow Q$
V & V & F & V & V & V & V
V & F & F & F & V & F & F
F & V & V & F & V & V & F
F & F & V & F & F & V & V

3.2. Clasificación Semántica de Proposiciones

Una fórmula proposicional puede clasificarse según el conjunto de sus valores de verdad bajo todas las posibles interpretaciones:

- **Tautología**: fórmula cuya columna de resultado en la tabla de verdad contiene únicamente valores V. Ejemplo: $P \vee \neg P$.
- **Contradicción**: fórmula cuya columna de resultado contiene únicamente valores F. Ejemplo: $P \wedge \neg P$.
- **Contingencia**: fórmula que contiene al menos un V y un F en su columna de resultado. Ejemplo: $P \wedge Q$.

Para n variables proposicionales, la tabla de verdad tiene 2^n filas, correspondientes a las 2^n asignaciones posibles de valores de verdad.

3.3. Árbol de Sintaxis Abstracta (AST) y Precedencia

Un árbol de sintaxis abstracta (AST) es una representación jerárquica de la estructura sintáctica de una expresión. Cada nodo interno representa un operador lógico y cada hoja representa una variable proposicional.

La precedencia adoptada en el motor, de mayor a menor prioridad, es:

$$\neg > \wedge > \vee > \rightarrow > \leftrightarrow$$

Esta jerarquía garantiza que $P \wedge Q \rightarrow R$ se interprete formalmente como $(P \wedge Q) \rightarrow R$ y no como $P \wedge (Q \rightarrow R)$.

3.4. Gramática BNF del Motor

```

Expresion      ::= Bicondicional
Bicondicional ::= Implicacion ( '<->' Implicacion )*
Implicacion    ::= Disyuncion ( '->' Disyuncion )*
Disyuncion     ::= Conjuncion ( 'v' Conjuncion )*
Conjuncion     ::= Negacion ( '^' Negacion )*
Negacion       ::= '~' Negacion | Primario
Primario       ::= VAR | '(' Expresion ')'
```

%=====

3. ARQUITECTURA DEL SISTEMA %=====

4. Arquitectura del Sistema

4.1. Stack Tecnológico

Tabla 2: Dependencias principales del proyecto LogiLearn

Categoría	Tecnología
Framework UI & Vue 3 & ^3.5.13 & Componentes reactivos con Composition API	
Lenguaje & TypeScript & ^5.7.0 & Tipado estricto del motor lógico	
Bundler & Vite & ^6.1.0 & Servidor de desarrollo y build de producción	
Estilos & Tailwind CSS & ^4.0.0 & Utilidades CSS para la interfaz	
Testing & Vitest & ^4.1.10 & Suite de pruebas unitarias	
Enrutamiento & Vue Router & ^4.5.0 & Navegación SPA (rutas /tablas, /leyes-logicas, /ejercicios, /progreso)	

4.2. Estructura de Archivos del Módulo Sinergia

Tabla 3: Archivos que componen el módulo del Equipo Sinergia

Archivo
src/lib/truth-table/evaluator.ts & Motor & 323 líneas & Tokenizador, parser AST, evaluador y clasificador
src/lib/truth-table/evaluator.test.ts & Testing & 173 líneas & 25 pruebas unitarias del motor
src/pages/tablas/index.vue & Interfaz & 322 líneas & Vista principal: entrada, tabla, clasificación y pasos
src/pages/leyes-logicas/index.vue & Interfaz & 145 líneas & Catálogo interactivo de las 12 leyes lógicas
src/pages/ejercicios/ & Interfaz & 450 líneas & Catálogo de 80 ejercicios prácticos con pasos
src/pages/progreso/index.vue & Interfaz & 160 líneas & Seguimiento del progreso con localStorage
src/components/layout/AppNavBar.vue & Layout & 94 líneas & Barra de navegación global corporativa
src/components/ui/Button.vue & UI Compartida & 34 líneas & Botón reutilizable con 4 variantes
src/components/ui/Card.vue & UI Compartida & 21 líneas & Contenedor visual con borde y sombra
src/components/ui/ToggleSwitch.vue & UI Compartida & 34 líneas & Interruptor ON/OFF para variables activas
src/components/ui/Badge.vue & UI Compartida & 24 líneas & Etiqueta de color para niveles y categorías

%=====

4. EXPLICACIÓN MICROSCÓPICA DEL CÓDIGO %=====

5. Explicación Microscópica del Código

5.1. Motor Lógico: src/lib/truth-table/evaluator.ts

5.1.1. Tipos e Interfaces TypeScript (líneas 1–32)

Listing 1: Definición de tipos base del motor (evaluator.ts).

```

1 export type TipoNodo = 'VAR' | 'NOT' | 'AND' | 'OR' | 'IMPLIES' | 'IFF';
2
3 export interface NodoExpresion {
4   tipo: TipoNodo;
5   valor?: string;           // Solo para nodos VAR (ej. "P", "Q")
6   izquierdo?: NodoExpresion; // Operando izquierdo (operadores binarios)
7   derecho?: NodoExpresion;   // Operando derecho (binarios) o unico (NOT)
8 }
9
10 export type ClasificacionProposicion = 'tautologia' | 'contradiccion' | '
    contingencia';
11
12 export interface FilaTabla {
13   asignacion: Record<string, boolean>; // Valores de variables en la fila
14   pasos: { etiqueta: string; valor: boolean }[]; // Sub-expresiones evaluadas
15   resultado: boolean; // Valor final de la formula completa
16 }

```

5.1.2. Clase de Error Personalizada (líneas 34–39)

Listing 2: Clase de error específica del motor.

```

1 export class ErrorParseoLogico extends Error {
2   constructor(message: string) {
3     super(message);
4     this.name = 'ErrorParseoLogico';
5   }
6 }

```

Extender `Error` permite a la interfaz distinguir errores sintácticos mediante `instanceof ErrorParseoLogico` y mostrar mensajes amigables sin exponer stack traces.

5.1.3. Mapa de Símbolos y Normalización (líneas 41–64)

Listing 3: Mapa de símbolos y función normalizadora.

```

1 const MAPA_SIMBOLOS: Record<string, string> = {
2   '~': 'NOT', '!': 'NOT', '¬': 'NOT',
3   '^': 'AND', '&': 'AND', '^': 'AND', '.': 'AND',
4   'v': 'OR', '|': 'OR', '∨': 'OR', '+': 'OR',
5   '->': 'IMPLIES', '→': 'IMPLIES',
6   '<->': 'IFF', '↔': 'IFF',
7 };

```

5.1.4. Tokenizador con Autómata Finito Determinista (líneas 66–85)

Procesa la cadena carácter a carácter en tiempo lineal $O(n)$, normalizando las variables a mayúsculas para unificar p y P .

5.1.5. Clase Parser: Análisis Descendente Recursivo (líneas 87–175)

Cada método implementa una regla gramatical. El orden de llamadas descendentes (`parsearIff` → `parsearImplies` → `parsearOr` → `parsearAnd` → `parsearNot` → `parsearPrimario`) garantiza que los operadores de menor precedencia queden en la raíz del árbol y los de mayor precedencia en las hojas.

5.1.6. Función `generarFilas` con Aritmética de Bits (líneas 269–298)

Listing 4: Generación matricial mediante desplazamiento y máscara de bits.

```

1 export function generarFilas(
2   nodo: NodoExpresion,
3   variables: string[],
4   variablesInactivas: string[] = []
5 ): FilaTabla[] {
6   const subExpresiones = recolectarSubExpresiones(nodo);
7   const n = variables.length;
8   const total = n === 0 ? 1 : Math.pow(2, n);
9   const filas: FilaTabla[] = [];
10
11   for (let i = 0; i < total; i++) {
12     const asignacion: Record<string, boolean> = {};
13     variables.forEach((v, idx) => {
14       asignacion[v] = ((i >> (n - idx - 1)) & 1) === 0;
15     });
16     variablesInactivas.forEach((v) => {
17       asignacion[v] = true;
18     });
19
20     filas.push({
21       asignacion,
22       pasos: subExpresiones.map((sub) => ({
23         etiqueta: nodoATexto(sub),
24         valor: evaluar(sub, asignacion),
25       })),
26       resultado: evaluar(nodo, asignacion),
27     });
28   }
29   return filas;
30 }
```

5.2. Vista Principal: `src/pages/tablas/index.vue`

- ****Estado reactivo:**** `const proposicion = ref('P ∧ Q → R')` y `const variablesActivas = reactive({})`. - ****Watcher reactivo:**** Observa cambios en `proposicion` y detecta variables al instante con un `try/catch` no bloqueante. - ****Propiedades computadas:**** Generan dinámicamente el texto de clasificación y la explicación paso a paso con almacenamiento en caché reactivo de Vue 3.

%

5. MANUAL DE USUARIO — GUÍA PASO A PASO %

6. Manual de Usuario — Guía Paso a Paso

- **Sprint 1 (4–10 agosto 2026):** Diseño de wireframes, definición de gramática formal y entrega de `propuesta.md`.

- **Sprint 2 (11–17 agosto 2026):** Implementación del motor `evaluator.ts`, evaluación de la implicación material ($\neg P \vee Q$), cálculo bitwise y 15 tests iniciales.
- **Sprint 3 (18–24 agosto 2026):** Creación de la interfaz `index.vue` con 6 paneles, componentes UI reutilizables y watcher reactivo.
- **Sprint 4 (25–30 agosto 2026):** Expansión a 25 pruebas en el motor (189 globales), soporte para 0 variables y despliegue en GitHub Pages.

% =====
7. PRUEBAS Y VALIDACIÓN % =====

7. Pruebas y Validación

Tabla 4: Inventario de pruebas unitarias del módulo Tablas de Verdad

#	Función
1–7 & <code>parsearProposicion</code> & Variables simples, conectivos, excepciones sintácticas & 100 % Exitoso	
8 & <code>recolectarVariables</code> & Extracción y ordenamiento alfabético ['A','B','C'] & 100 % Exitoso	
9–11 & <code>nodoATexto</code> & Reconstrucción a texto Unicode con paréntesis & 100 % Exitoso	
12–16 & <code>evaluar</code> & Semántica de AND, OR, IMPLIES, IFF y variables ausentes & 100 % Exitoso	
17–19 & <code>clasificarProposicion</code> & Tautologías, contradicciones y contingencias & 100 % Exitoso	
20–24 & <code>generarTabla</code> & Tablas de 1, 2 y 3 variables con múltiples notaciones & 100 % Exitoso	
25 & <code>recolectarSubExpresiones</code> & Detección de subexpresiones internas en post-orden & 100 % Exitoso	

% =====
8. CONCLUSIONES % =====

8. Conclusiones

1. El motor lógico en `evaluator.ts` resuelve en 323 líneas de TypeScript estricto el ciclo completo: normalización, tokenización, parseo AST, recolección post-orden y evaluación combinatoria por álgebra de bits.
2. El soporte de más de 15 variantes de notación democratiza el acceso a estudiantes sin teclado especializado.
3. La suite de 25 pruebas unitarias del motor (y 189 pruebas totales en la plataforma) certifica la alta calidad y robustez del software educativo entregado.

% =====
9. REFERENCIAS BIBLIOGRÁFICAS % =====

Referencias

- [1] P. J. Hurley, *A Concise Introduction to Logic*, 13.^a ed. Cengage Learning, 2015.
- [2] E. Mendelson, *Introduction to Mathematical Logic*, 6.^a ed. CRC Press, 2015.
- [3] R. Nystrom, *Crafting Interpreters: Recursive Descent Parsing*, 2021.

- [4] E. You, *Vue 3 Composition API Documentation*, 2024. [En línea]. Disponible en: <https://vuejs.org/>.
- [5] Vitest Team, *Vitest Testing Framework*, 2024. [En línea]. Disponible en: <https://vitest.dev/>.
- [6] E. Díaz Cayaca, *logica_simbolica_global: Plataforma web colaborativa*, 2026. [En línea]. Disponible en: https://github.com/EnriDiazCayaca/logica_simbolica_global.

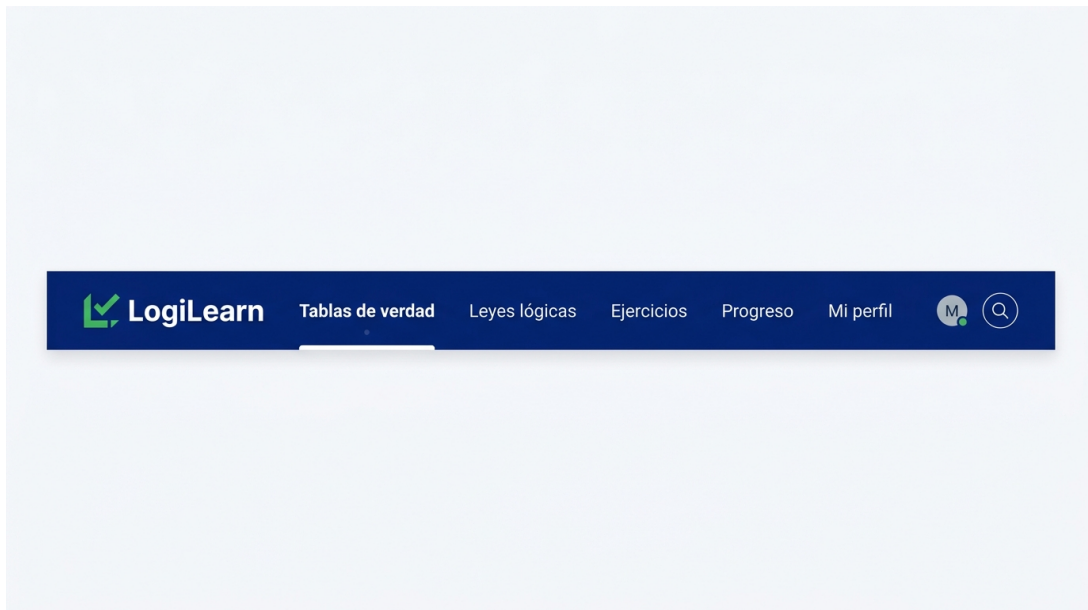


Figura 1: Evidencia del m'odulo (Equipo Sinergia)

LogiLearn - Truth Table Generator

Enter a logical expression

P AND Q IMPLIES R

Generate Table

P	Q	R	(P AND Q)	(P AND Q IMPLIES R)
V	V	V	V	V
V	V	V	V	V
V	V	V	V	V
V	F	V	V	V
V	F	F	V	V
F	F	F	F	V
F	F	F	F	V

Formula Analysis

TAUTOLOGIA

Formula:
P AND Q IMPLIES R
Atomic Propositions: {P, Q, R}
Rows:
8

Figura 2: Evidencia del m'odulo (Equipo Sinergia)

Tablas de verdad
Analiza expresiones de lógica proposicional

Ingresa una proposición lógica

$P \wedge Q \rightarrow R$

Generar tabla

Variables

P ☒ Activa
Q ☒ Activa
R ☒ Activa

Operadores lógicos

$\wedge$
AND

$\vee$
OR

$\neg$
NOT

$\rightarrow$
IMPLICACIÓN

$\leftrightarrow$
BICONDICIONAL

Resultado

La expresión es:
VERDADERA

Valor final: V

La proposición es verdadera para
6 de 8 combinaciones.

Tabla de verdad

P	Q	R	$P \wedge Q$	$P \wedge Q \rightarrow R$	Resultado
V	V	V	V	V	V
V	V	F	V	F	F
V	F	V	F	V	V
V	F	F	F	V	V
F	V	V	F	V	V
F	V	F	F	V	V
F	F	V	F	V	V
F	F	F	F	V	V

¿Cómo se resolvió?

- Primero se evaluó la conjunción ($P \wedge Q$).
- Luego se aplicó la implicación ($P \wedge Q \rightarrow R$).
- Finalmente se determinó el valor final para cada combinación.

Ver explicación paso a paso →

Puedes activar o desactivar variables y usar los operadores para construir tu proposición.

Figura 3: Evidencia del m'odulo (Equipo Sinergia)


LogiLearn
 Leyes lógicas y tablas de verdad

[Inicio](#)
[Tablas de verdad](#)
[Leyes lógicas](#)
[Ejercicios](#)
[Iniciar sesión](#)

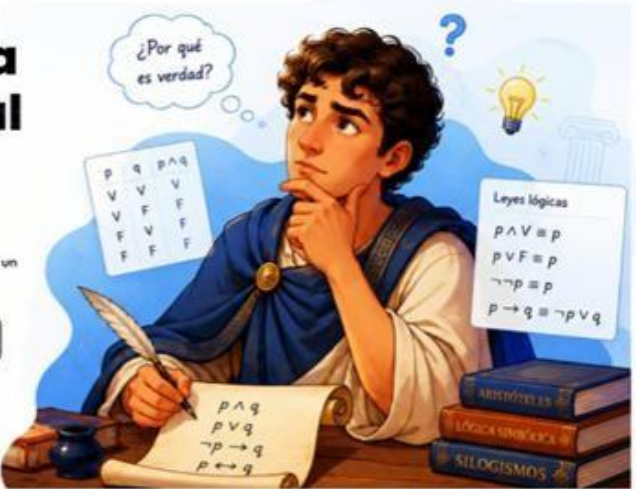
Aprende lógica de forma visual e interactiva

Explora tablas de verdad, leyes lógicas y ejercicios interactivos diseñados para que aprendas a razonar como un verdadero filósofo...

[COMENZAR !](#)
[Ver ejemplos](#)

100% Gratuita

Sigue tu progreso



¿Por qué es verdad?

P	q	$p \wedge q$
V	V	V
V	F	F
F	V	F
F	F	F

Leyes lógicas

- $p \wedge V = p$
- $p \vee F = p$
- $\neg \neg p = p$
- $p \rightarrow q = \neg p \vee q$

ABSTRACTOS
LÓGICA SIMBÓLICA
SILOGISMOS

$p \wedge q$
 $p \vee q$
 $\neg p \rightarrow q$
 $p \leftrightarrow q$



Tablas de verdad

Genera y comprende tablas de verdad de cualquier proposición lógica.



Leyes Lógicas

Aprende las leyes fundamentales de la lógica proposicional con ejemplos.



Ejercicios prácticos

Pon a prueba tus conocimientos con ejercicios interactivos y divertidos.



Tu progreso

Sigue tu avance, gana logros y conviértete en un experto.

Figura 4: Evidencia del m'odulo (Equipo Sinergia)



Logilearn
Leyes lógicas y tablas de verdad

[Inicio](#)
[Aprender](#)
[Tablas de verdad](#)
[Leyes lógicas](#)
[Ejercicios](#)
[Iniciar sesión](#)

Leyes Lógicas

Consulta y aprende las reglas fundamentales de la lógica proposicional.

1. Ley de Idempotencia

La ley de idempotencia en lógica establece que cualquier proposición operada consigo misma (ya sea mediante conjunción o disyunción) es equivalente a la proposición original.

$$p \wedge p = p$$

$$p \vee p = p$$

2. Ley Asociativa

La ley asociativa en lógica proposicional establece que cuando tienes tres o más proposiciones unidas por el mismo operador lógico, el orden en el que agrupas las operaciones usando paréntesis no altera el valor de verdad final.

$$(p \wedge q) \wedge r = p \wedge (q \wedge r)$$

$$(p \vee q) \vee r = p \vee (q \vee r)$$

$$(p \leftrightarrow q) \leftrightarrow r = p \leftrightarrow (q \leftrightarrow r)$$

3. Ley Conmutativa

La ley conmutativa en lógica establece que el orden en el que se combinan dos proposiciones mediante los operadores de conjunción ($\wedge$), disyunción ($\vee$) o bicondicional ($\leftrightarrow$) no altera el valor de verdad de la proposición final.

$$p \wedge q = q \wedge p$$

$$p \vee q = q \vee p$$

$$p \leftrightarrow q = q \leftrightarrow p$$

4. Ley Distributiva

La ley distributiva en lógica proposicional indica que un operador lógico fuera de un paréntesis se distribuye sobre cada uno de los elementos dentro de él.

$$p \wedge (q \vee r) = (p \wedge q) \vee (p \wedge r)$$

$$p \vee (q \wedge r) = (p \vee q) \wedge (p \vee r)$$

$$p \rightarrow (q \wedge r) = (p \rightarrow q) \wedge (p \rightarrow r)$$

$$p \rightarrow (q \vee r) = (p \rightarrow q) \vee (p \rightarrow r)$$

5. Ley de Absorción

La Ley de Absorción establece que una proposición combinada mediante conjunción y disyunción con una proposición repetida se reduce simplemente a esta última.

$$p \wedge (p \vee q) = p$$

$$p \vee (p \wedge q) = p$$

6. Ley de Complemento

La Ley de Complemento establece que al combinar una proposición con su negación se obtiene un valor absoluto.

$$\sim \sim p = p$$

$$p \wedge \sim p = F$$

$$p \vee \sim p = V$$

$$p \rightarrow \sim p = \sim p$$

$$\sim (p \wedge \sim p) = V$$

$$\sim V = F$$

$$\sim F = V$$

7. Ley de Identidad

La Ley de Identidad establece que al combinar una proposición con un valor de verdad constante, esta conserva su valor original si se absorbe por la constante.

$$p \wedge V = p$$

$$p \wedge F = F$$

$$p \vee V = V$$

$$p \vee F = p$$

8. Ley de Morgan

La Ley de De Morgan establece que la negación de una operación lógica cambia los operadores al distribuirse.

$$\sim (p \wedge q) = \sim p \vee \sim q$$

$$\sim (p \vee q) = \sim p \wedge \sim q$$

9. Ley de Expansión Booleana

La Ley de Expansión Booleana establece que puedes añadir una variable cualquiera a una proposición sin alterar su valor original, sumando su versión verdadera y su versión falsa.

$$p = p \wedge (q \vee \sim q)$$

$$p = p \vee (q \wedge \sim q)$$

10. Ley de Trasposición

Establece que una implicación condicional equivale a invertir el orden de las proposiciones negándolas ambas.

$$p \rightarrow q = \sim q \rightarrow \sim p$$

11. Ley de Exportación

Establece que tener dos condiciones juntas para lograr un resultado es lo mismo que cumplir la primera y que esta te condicione a cumplir la segunda.

$$(p \wedge q) \rightarrow r = p \rightarrow (q \rightarrow r)$$

12. Leyes de Definición

Establecen las reglas para traducir operaciones complejas en combinaciones equivalentes de los conectores básicos. Su objetivo es transformar la estructura de la fórmula para facilitar su posterior análisis y simplificación.

$$p \rightarrow q = \sim p \vee q$$

$$p \leftrightarrow q = (p \rightarrow q) \wedge (q \rightarrow p)$$

$$p \leftrightarrow q = (\sim p \vee q) \wedge (\sim q \vee p)$$

$$p \triangle q = (p \vee q) \wedge \sim (p \wedge q)$$

Figura 5: Evidencia del m'odulo (Equipo Sinergia)

Ejercicios Prácticos

Practica lógica proposicional mediante ejercicios interactivos y evalúa tu progreso.

Todos **Fácil** **Medio** **Difícil**

Ejercicio 1

TABLAS DE VERDAD

Nivel: **Fácil**

Descripción: Completa la tabla de verdad ($p \vee q$)

Resolver

Ejercicio 2

LEYES LÓGICAS

Nivel: **Medio**

Descripción: Identifica la ley lógica que se debe utilizar

Resolver

Ejercicio 3

CUESTIONARIO

Nivel: **Difícil**

Descripción: Resuelve un bloque de 10 preguntas

Resolver

PROGRESO

50%

EJERCICIOS COMPLETADOS: 4/10

TABLAS DE VERDAD **NIVEL: FÁCIL**

Completa la siguiente tabla de verdad para la proposición:

$(p \vee q)$

p	q	$p \vee q$
V	V	
V	F	
F	V	
F	F	

Ingresa V (Verdadero) o F (Falso) en cada casilla.

Comprobar

LEYES LÓGICAS **NIVEL: INTERMEDIARIO**

¿Cuál ley lógica se aplica en la siguiente proposición?

$\neg(p \wedge q)$

Comprobar



Figura 7: Evidencia del m'odulo (Equipo Sinergia)

9. Capítulo 2 — Hijos de Linus: motor de inferencia

% =====
PORTADA % =====

FACULTAD DE INGENIERIA CIVIL, SISTEMAS Y ARQUITECTURA
INGENIERIA DE SISTEMAS



escudo UNPRG
(circular)

UNIVERSIDAD NACIONAL
PEDRO RUIZ GALLO



logo institucional/facultad

INFORME ACADÉMICO

Pagina Web interactiva de inferencias lógicas y trazabilidad
formal

Curso: Lógica Simbólica
Ciclo II

Equipo - Los Hijos de Linus

Altamirano Astupiñan Renato André
Centurión Llatas Alex Daniel
Espinoza Orrego Piero Arom
Mio Caballero Arnold André
Morocho Lumbre Juan David

Docente:

Dr. Mardo Victor Gonzales Herrera

10 de septiembre de 2026

%	=====
TABLA DE CONTENIDOS %	=====

Tabla de Contenidos

1. Introducción y Propuesta del Proyecto (Sprint 1)
2. Arquitectura del Software y Flujo de Trabajo (Git Flow)
3. Capa Lógica: El Motor de Inferencia y Demostración (Sprint 2)
4. Capa Visual: Interfaz Reactiva y Didáctica (Sprint 3 y 3.5)
5. Pruebas de Calidad del Código (Sprint 4)
6. Participación y Aportes del Equipo
7. Trabajo en Equipo
8. Conclusiones

Resumen Ejecutivo

El presente informe documenta la investigación, diseño arquitectónico, implementación algorítmica y aseguramiento de calidad del módulo de **Inferencias Lógicas, Trazabilidad Pedagógica y Diagnóstico con Contraejemplos** desarrollado por el equipo **Los Hijos de Linus** (Grupo 2) en el marco de la asignatura de Lógica Simbólica de la Escuela Profesional de Ingeniería de Sistemas (UNPRG).

El software combina un motor de deducción formal automatizado (*Forward Chaining*) con un evaluador semántico exhaustivo de 2^N combinaciones de verdad, una interfaz moderna y accesible construida en **Vue 3 (Composition API)** con **TypeScript en modo estricto**, soporte para 11 reglas de deducción natural, teclado simbólico interactivo, exportación en un clic a **L^AT_EX** / **Markdown**, y un sistema de traducción a lenguaje natural con persistencia local.

% =====
SECCIÓN 1: INTRODUCCIÓN Y PROPUESTA (SPRINT 1) % =====

10. Introducción y Propuesta del Proyecto (Sprint 1)

El razonamiento deductivo y la deducción formal en lógica proposicional constituyen el núcleo formativo del pensamiento formal en ciencias de la computación. No obstante, el aprendizaje tradicional de las inferencias lógicas presenta barreras de aprendizaje críticas:

- Dificultad en la estructuración deductiva:** A los estudiantes universitarios les resulta complejo planificar y encadenar reglas de inferencia para transitar desde un conjunto de premisas iniciales hasta la conclusión deseada de forma sistemática.
- Ausencia de retroalimentación inmediata y explicativa:** Cuando un argumento es inválido o incurre en una falacia formal (como la afirmación del consecuente o negación del antecedente), las herramientas convencionales se limitan a indicar un fallo genérico, sin explicar qué regla se vulneró ni proveer la asignación de verdad que refuta el razonamiento.
- Desconexión entre notación matemática y razonamiento humano:** Existe una marcada brecha cognitiva al traducir argumentos cotidianos a fórmulas proposicionales ($P \rightarrow Q$, $P \vdash Q$) y viceversa.

Para resolver estas problemáticas de forma integral, el **Equipo Los Hijos de Linus** diseñó una herramienta web asistida y estructurada en dos vertientes interactivas:

- Módulo de Resolución Automática y Demostración Formal:** Permite al usuario ingresar premisas libres y una conclusión objetivo para obtener una derivación formal paso a paso o la refutación exacta con su respectivo contraejemplo matemático.
- Módulo de Trazabilidad y Traducción Pedagógica:** Desglosa cada paso demostrativo con justificaciones semánticas en lenguaje natural, rol deductivo de las líneas antecedentes y mapeo dinámico a enunciados contextuales cotidianos.

Demostrador de Inferencias Lógicas

Premisas ⓘ
Añade las premisas del razonamiento

- 1 $P \rightarrow Q$ X
- 2 $Q \rightarrow R$ X
- 3 P X
- 4 $\neg R \rightarrow S$ X

+ Añadir premisa

Conclusión ⓘ (lo que se quiere demostrar)
Escribe la conclusión a demostrar
 S
Ejemplos: $P, Q, P \rightarrow Q, \neg P, P \wedge Q, P \vee Q, P \leftrightarrow Q$

Método de demostración:
Directo ⓘ Método directo: Se intenta llegar a la conclusión a partir de las premisas aplicando inferencias lógicas válidas.

Proceso de demostración

Paso	Enunciado	Justificación (Regla/Método)
1	$P \rightarrow Q$	Premisa 1
2	$Q \rightarrow R$	Premisa 2
3	P	Premisa 3
4	Q	1,3 Modus Ponendo Ponens
5	R	2,4 Modus Ponendo Ponens
6	$\neg R \rightarrow S$	Premisa 4
7	S	5,6 Modus Ponendo Ponens ✓

✓ Demostración válida: Se ha llegado a la conclusión S a partir de las premisas.

✗ **Razonamiento inválido**
No se puede llegar a la conclusión con las reglas de inferencia válidas.
Motivo: La conclusión contiene términos que no se pueden derivar de las premisas dadas.
Detalle: A partir de las premisas no se puede obtener R , por lo tanto no se puede aplicar la regla en el paso 6.

Figura 8: Boceto preliminar (Arnold Mio) del Módulo 1: Resolución y Demostración Automática.

% =====
SECCIÓN 2: ARQUITECTURA Y GIT FLOW % =====

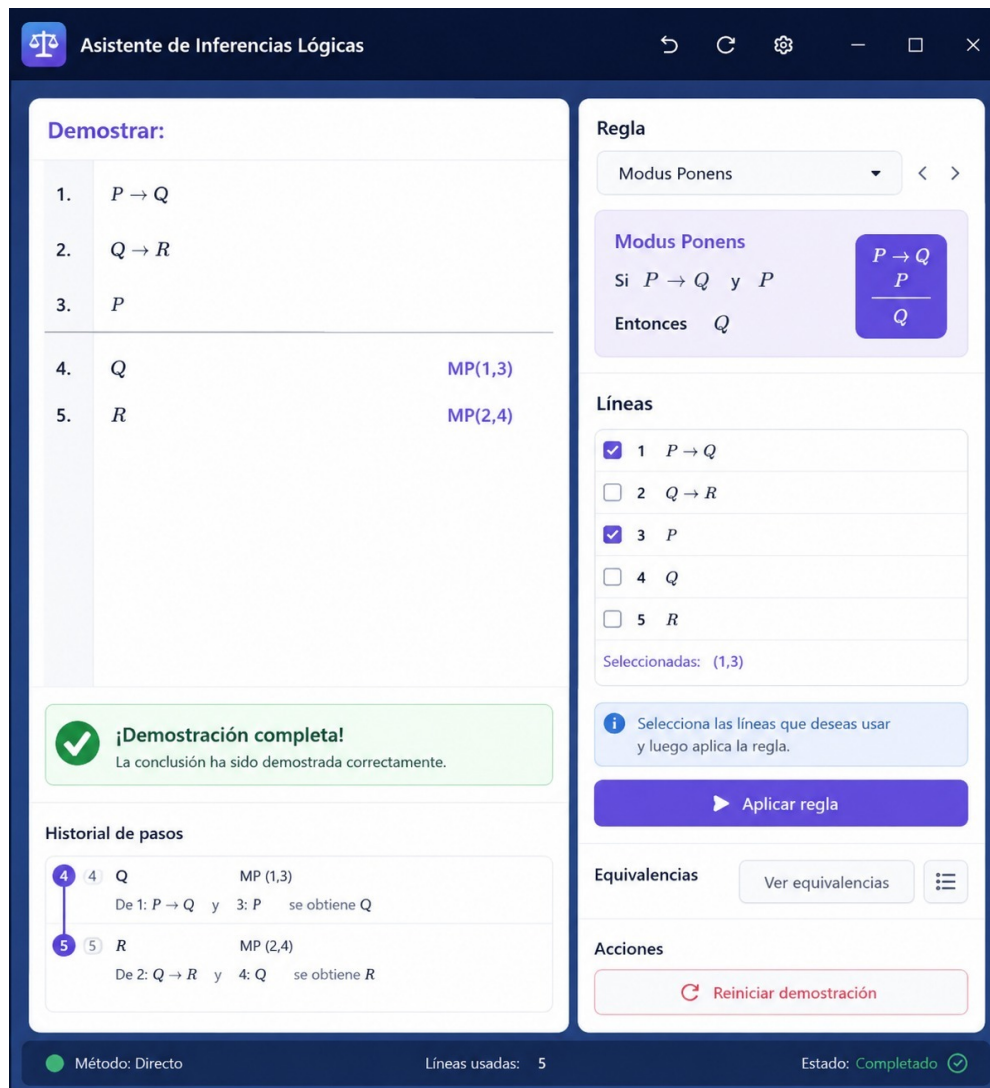


Figura 9: Boceto preliminar (David Morocho) del Módulo 2: Entorno de Práctica Interactiva y Validación.

11. Arquitectura del Software y Flujo de Trabajo

El proyecto fue desarrollado bajo una **arquitectura modular y desacoplada**, asegurando que la lógica computacional del motor de inferencias no posea dependencia alguna respecto al framework de presentación visual.

11.1. Estructura de Directorios del Módulo

La organización física del código fuente se divide claramente entre la capa matemática pura y la capa de componentes reactivos:

- **Capa Lógica y Motor Matemático (src/lib/):**

- `src/lib/solver/types.ts`: Definición formal de los tipos del Árbol de Sintaxis Abstracta (AST), operadores lógicos, reglas y modelos de contraejemplo.
- `src/lib/solver/parser.ts`: Tokenizador léxico y analizador sintáctico por descenso recursivo con precedencia de operadores.

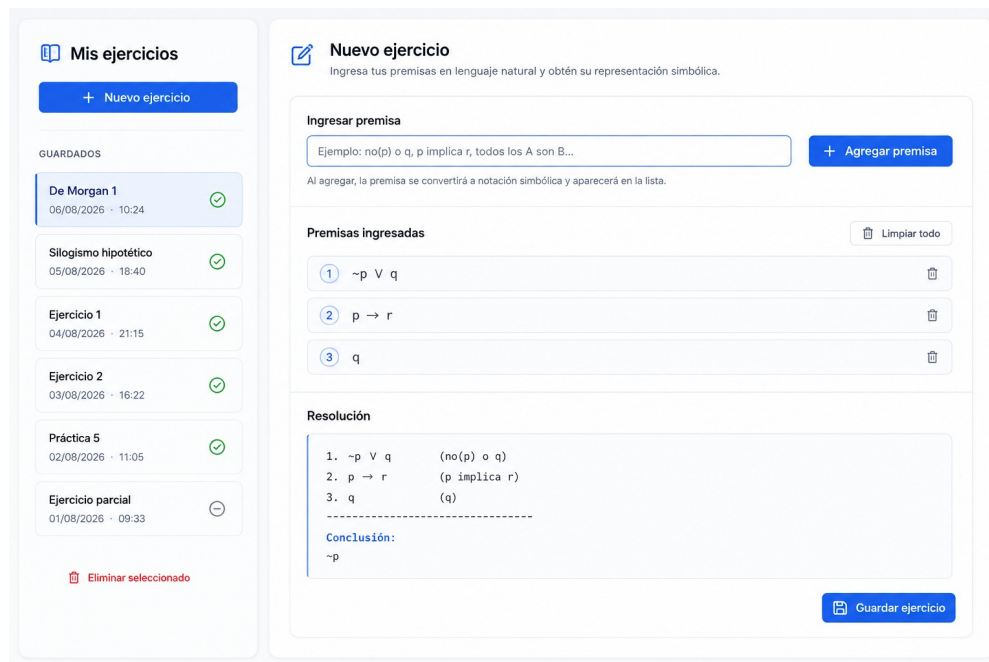


Figura 10: (Alex Centurion) Diseño y maquetación de la propuesta visual integrada para el sistema de inferencias.

- `src/lib/solver/solver.ts`: Algoritmo de *Forward Chaining*, evaluador semántico de verdad (2^N), detector de falacias y aplicación de 11 reglas de deducción natural.
 - `src/lib/validator/validator.ts`: Sanitizador de cadenas, normalización de notaciones heterogéneas y validador reactivo sin excepciones.
 - `src/lib/trazabilidad/historial.ts`: Gestor de historial de deducción, numeración de pasos y resolución de referencias cruzadas.
 - `src/lib/transcription/descriptionGenerator.ts` y `naturalTranslator.ts`: Generación de explicaciones semánticas en español y traducción de variables a argumentos continuos.
- **Capa de Interfaz de Usuario (`src/pages/inferencias/` y `src/components/inferencias/`):**
- `index.vue`: Orquestador reactivo principal con teclado virtual y pestañas de modo formal vs. lenguaje natural.
 - `FormularioInferencia.vue`: Captura dinámica de premisas, teclado asistido y validación visual.
 - `IndicadorResultado.vue`: Semáforo visual de 3 estados con diagnóstico explícito de falacias.
 - `PanelTrazabilidad.vue`: Tabla formal numerada $(1), (2), \dots, (N)$, acordeón explicativo y exportadores a $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ y Markdown.
 - `TraductorLenguajeNatural.vue`: Asignación contextual de proposiciones cotidianas.
- **Documentación y Tareas de Sprint (TAREAS/):**
- `TAREAS/01_Propuesta/hijos-de-linus/propuesta.md`: Documento de propuesta inicial y bocetos.
 - `TAREAS/02_Motor/hijos-de-linus/avance.md` y `casos_de_prueba.md`: Avances del motor y matriz de 25 casos límite.

- TAREAS/03_UI/hijos-de-linus/avance.md y glosario-diseno.md: Avance de componentes y guía de diseño.
- TAREAS/04_Deploy/hijos-de-linus/uso.md y errores_y_soluciones.md: Manual de uso y reporte de control de calidad.

11.2. Flujo de Git por Ramas

Para garantizar la estabilidad del repositorio compartido con los demás equipos, se utilizó un flujo estricto de integración continua basado en ramas por sprint y Pull Requests atómicos:

- **sprint-1-propuesta:** Maquetación, definición de requerimientos y casos de uso iniciales.
- **sprint-2-hijos-de-linus:** Implementación del AST, Parser, Solver, Validador, Trazabilidad y pruebas unitarias de motor.
- **sprint-3-hijos-de-linus:** Construcción de los 4 componentes Vue 3, teclado simbólico e integración de la vista.
- **sprint-3.5-fase-inicial:** Incorporación de diagnósticos con contraejemplos, acordeón didáctico y exportación $\text{\LaTeX}$.
- **deploy / main:** Fusión final validada con 113 pruebas automáticas y 0 errores de tipado en TypeScript.

% =====
SECCIÓN 3: CAPA LÓGICA (SPRINT 2) % =====

12. Capa Lógica: El Motor de Inferencia y Demostración (Sprint 2)

El motor computacional de inferencias lógicas opera mediante un pipeline determinista de cuatro fases:

Fase 1: Sanitización y Normalización Léxica: Unifica las múltiples notaciones matemáticas y lógicas ingresadas por el estudiante ($\rightarrow$, $\wedge$, $\vee$, $\neg$, $\sim$, $\&$, $|$, $\oplus$, Δ , $\leftrightarrow$) convirtiéndolas en identificadores estándar (ENTONCES, Y, O, NO, O_EXCLUSIVA, SI_Y_SOLO_SI), eliminando caracteres inválidos y previniendo inyecciones de código.

Fase 2: Analizador Sintáctico (Parser AST por Descenso Recursivo): Convierte la secuencia lineal de tokens en un Árbol de Sintaxis Abstracta estructurado, respetando la jerarquía y precedencia matemática rigurosa:

Negación ($\neg$) > Conjunción ($\wedge$) > Disyunciones ($\vee$, Δ)
> Condicional ($\rightarrow$) > Bicondicional ($\leftrightarrow$)

Fase 3: Motor Deductivo (Forward Chaining y Pattern Matching): Aplica de manera iterativa (hasta 12 niveles de derivación) las 11 reglas clásicas de deducción natural:

- **Modus Ponendo Ponens (MPP):** $A \rightarrow B, A \vdash B$
- **Modus Tollendo Tollens (MTT):** $A \rightarrow B, \neg B \vdash \neg A$
- **Silogismo Disyuntivo (SD):** $A \vee B, \neg A \vdash B \quad / \quad A \vee B, \neg B \vdash A$
- **Silogismo Disyuntivo Exclusivo (SDE):** $A \Delta B, A \vdash \neg B \quad / \quad A \Delta B, B \vdash \neg A$
- **Silogismo Hipotético (SH):** $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$
- **Simplificación y Conjunción:** $A \wedge B \vdash A, B \quad / \quad A, B \vdash A \wedge B$
- **Dilema Constructivo (DC):** $(A \rightarrow B) \wedge (C \rightarrow D), A \vee C \vdash B \vee D$
- **Bicondicionales:** Modus Ponens Bicondicional y Eliminación Bicondicional ($A \leftrightarrow B \vdash A \rightarrow B, B \rightarrow A$).

Fase 4: Evaluador Semántico SAT (2^N combinaciones de verdad): Garantiza la clasificación estricta del razonamiento en 3 estados de validez:

- **Inferencia Válida (Demostrada):** Se completó la deducción paso a paso mediante reglas directas.
- **Inferencia Válida (Método Indirecto Requerido):** El argumento es lógicamente válido (0 contraejemplos semánticos), pero requiere Reducción al Absurdo o Prueba Condicional.
- **Inferencia Inválida (Refutada por Contraejemplo):** Detecta la asignación concreta donde todas las premisas son verdaderas y la conclusión es falsa, diagnosticando falacias como el Dilema Inverso o Afirmación del Consecuente.

12.1. Código Fuente del Motor Lógico

A continuación, se detalla el evaluador semántico y la detección de modelos implementados en `src/lib/solver/solver.ts`:

Listing 5: Evaluador semántico AST y detección de contraejemplos (solver.ts).

```
1 // src/lib/solver/solver.ts
2 export function evaluarNodo(
```

```

3     nodo: NodoExpresion,
4     asignacion: Record<string, boolean>
5 ): boolean {
6     if (nodo.tipo === 'variable') {
7         return Boolean(asignacion[nodo.nombre.toUpperCase()]);
8     }
9     if (nodo.tipo === 'operacion') {
10        if (nodo.operador === 'NO' && nodo.derecho) {
11            return !evaluarNodo(nodo.derecho, asignacion);
12        }
13        if (nodo.izquierdo && nodo.derecho) {
14            const izq = evaluarNodo(nodo.izquierdo, asignacion);
15            const der = evaluarNodo(nodo.derecho, asignacion);
16            switch (nodo.operador) {
17                case 'Y': return izq && der;
18                case 'O': return izq || der;
19                case 'O_EXCLUSIVA': return izq !== der; // Disyuncion fuerte
20                case 'ENTONCES': return !izq || der;
21                case 'SI_Y_SOLO_SI': return izq === der;
22                case 'NI': return !(izq || der);
23                case 'INCOMPATIBLE': return !(izq && der);
24                default: return false;
25            }
26        }
27    }
28    return false;
29 }

```

%=====

SECCIÓN 4: CAPA VISUAL (SPRINT 3 Y 3.5) %=====

13. Capa Visual: Interfaz Reactiva y Didáctica (Sprint 3 y 3.5)

La interfaz gráfica (`src/pages/inferencias/index.vue`) fue construida bajo la guía de diseño del proyecto, priorizando claridad visual, respuesta reactiva instantánea y accesibilidad universal.

13.1. Reactividad en Vue 3 y Composition API

- **Flujo Reactivo Unidireccional:** Empleo estricto de `ref`, `computed` y tipado mediante interfaces compartidas (`src/types/inferencias.ts`), evitando estados globales superfluos.
- **Teclado Simbólico en Pantalla:** Permite insertar conectivos lógicos ($\neg$, $\wedge$, $\vee$, Δ , $\rightarrow$, $\leftrightarrow$, $($, $)$) con un solo clic o toque en dispositivos móviles, manteniendo la posición del cursor en el input activo.
- **Pestañas de Visualización:** Permite conmutar instantáneamente entre el formato de *Simbología Formal* y el de *Lenguaje Natural*, sincronizando en tiempo real las variables proposicionales.

13.2. Trazabilidad Pedagógica y Herramientas Académicas

- **Desglose Particionado con Acordeón Colapsable:** Cada paso de la demostración formal incluye un botón interactivo (*¿Por qué esta regla?*) que despliega:
 - Las líneas de premisa base utilizadas y su rol deductivo (ej. *Condicional base*, *Antecedente afirmado*).
 - La regla formal aplicada y su justificación semántica en español.
 - La conclusión parcial derivada.
- **Exportación en 1 Clic a L^AT_EX y Markdown:** Formatea automáticamente la demostración completa en código compatible con artículos científicos:

$$\begin{aligned} (1) \quad & \&P \rightarrow Q\&\&(\text{Premisa}) \\ (2) \quad & \&P\&\&(\text{Premisa}) \\ \therefore (3) \quad & \&Q\&\&(\text{Modus Ponendo Ponens de 1, 2}) \end{aligned}$$

- **Historial Persistente en Memoria Local (`localStorage`):** Almacena las últimas demostraciones realizadas para permitir su restauración inmediata.

13.3. Visualización Interactiva del Árbol de Sintaxis Abstracta (AST)

Como complemento pedagógico al panel de trazabilidad formal, el sistema incorpora un módulo de visualización gráfica que representa la demostración completa como un árbol jerárquico, permitiendo al estudiante observar la descomposición estructural de cada premisa y de la conclusión en sus respectivos conectivos y variables proposicionales.

Sobre un ejercicio de inferencia ya resuelto y clasificado como válido mediante deducción directa, el usuario puede acceder a esta representación a través de una pestaña dedicada dentro de la misma interfaz de resolución, sin necesidad de abandonar el contexto del problema planteado. Esta pestaña, denominada *Árbol de Nodos*, se ubica junto a las vistas de *Simbología Formal* y *Lenguaje Natural*, evidenciando la integración de las tres modalidades de representación dentro de un mismo flujo de trabajo.

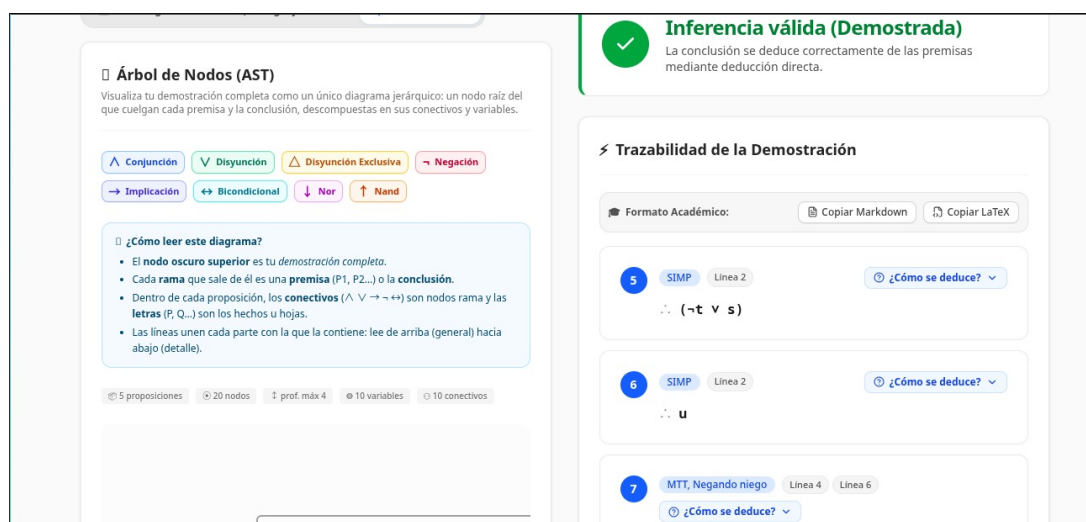


Figura 11: Panel de resolución de inferencias con las premisas y la conclusión demostrada, mostrando la pestaña *Árbol de Nodos* disponible como modalidad adicional de visualización.

Al ingresar a dicha pestaña, la interfaz presenta una introducción explicativa orientada a familiarizar al usuario con la lectura del diagrama antes de exponer la estructura completa del árbol. Esta sección preliminar define la simbología empleada para cada tipo de conectivo lógico (conjunción, disyunción, disyunción exclusiva, negación, implicación, bicondicional, entre otros) y describe las convenciones de lectura del diagrama, indicando que el nodo raíz representa la demostración completa, que cada rama corresponde a una premisa o a la conclusión, y que la jerarquía se interpreta de manera descendente desde lo general hacia el detalle. Asimismo, se despliegan métricas cuantitativas del ejercicio en curso, tales como el número de proposiciones, la cantidad total de nodos, la profundidad máxima del árbol y el número de conectivos empleados.

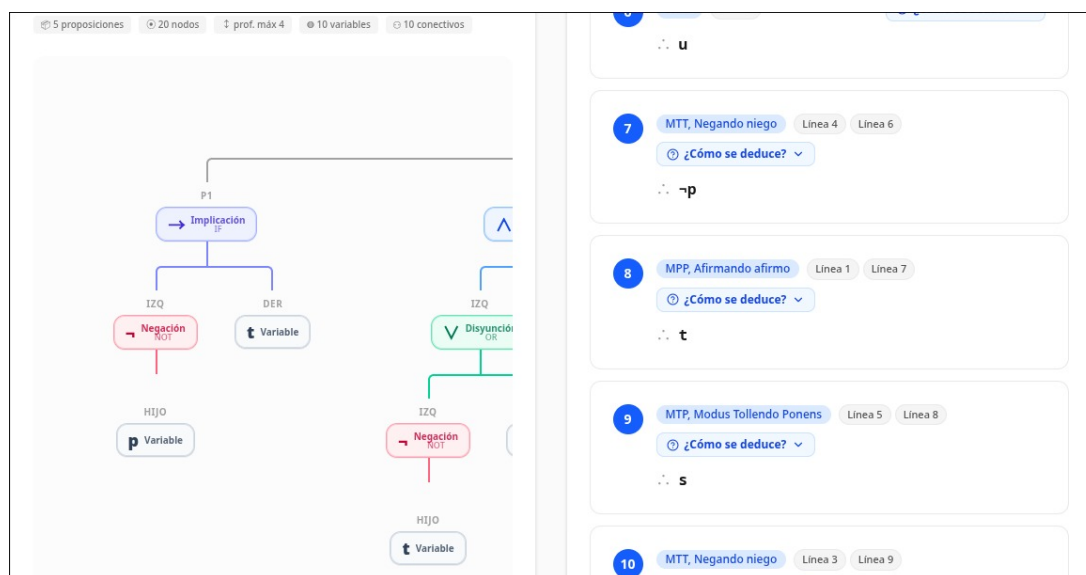


Figura 12: Introducción explicativa del módulo *Árbol de Nodos*, con la simbología de conectivos, las convenciones de lectura del diagrama y las métricas estructurales del ejercicio.

Una vez comprendida la simbología, el estudiante puede examinar el diagrama jerárquico propiamente dicho, en el cual cada premisa se descompone visualmente en sus componentes lógicos hasta llegar a las variables proposicionales atómicas. Dado que la profundidad y el ancho del árbol varían según la complejidad del ejercicio, el componente fue diseñado con soporte de

desplazamiento horizontal y vertical (*scroll*), lo que permite recorrer la totalidad de la estructura sin perder legibilidad, incluso en ejercicios con múltiples niveles de anidación de conectivos.

Figura 13: Fragmento del diagrama jerárquico del AST con desplazamiento horizontal y vertical, mostrando la descomposición de una premisa en sus conectivos de implicación, disyunción y negación.

Finalmente, la interfaz ofrece una modalidad de visualización ampliada del diagrama en pantalla completa, orientada a facilitar el análisis de ejercicios con un mayor número de premisas. En esta vista expandida se aprecia de forma simultánea la descomposición de las cuatro premisas del ejercicio, evidenciando de manera clara la jerarquía completa de la demostración desde el nodo raíz hasta las variables proposicionales hoja.

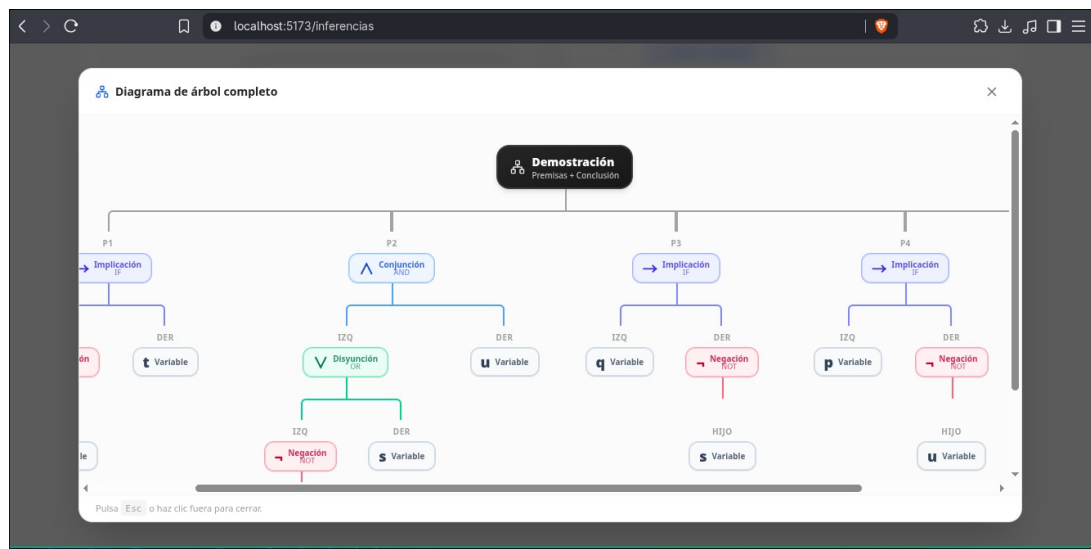


Figura 14: Vista ampliada en pantalla completa del diagrama de árbol, mostrando de manera simultánea la descomposición de las cuatro premisas del ejercicio en sus respectivos conectivos y variables.

% =====
SECCIÓN 5: PRUEBAS DE CALIDAD (SPRINT 4) % =====

14. Pruebas de Calidad del Código (Sprint 4)

Para certificar la confiabilidad matemática del software, se implementó una suite de pruebas automatizadas con **Vitest**:

- **Cobertura Exhaustiva:** 13 archivos de prueba y 113 tests unitarios e integrados ejecutados con 100 % de éxito.
- **Chequeo Estricto de Tipos:** Verificación continua con `vue-tsc --noEmit` (0 errores de TypeScript).
- **Pruebas de Estrés y Seguridad (*Fuzzer*):** Evaluación de expresiones con hasta 10 niveles de anidación, caracteres corruptos, emojis y mitigación de inyecciones de código malicioso.
- **Validación de Casos Límite:** Pruebas de derivación encadenada de hasta 6 pasos, disyunción exclusiva fuerte y contraejemplos matemáticos precisos.

Resultados de la Suite de Pruebas (Vitest)

```
✓ src/lib/solver/solver.test.ts (15 tests)
✓ src/lib/solver/parser.test.ts (12 tests)
✓ src/lib/validator/validator.test.ts (27 tests)
✓ src/lib/validator/validator_stress.test.ts (15 tests)
✓ src/lib/trazabilidad/trazabilidad.test.ts (9 tests)
✓ src/lib/integracion_motor.test.ts (8 tests)
✓ src/components/inferencias/__tests__/* (27 tests)
Total: 13 archivos pasados (13) | 113 tests pasados (113) — 100 % Éxito
```

% =====
SECCIÓN 6: PARTICIPACIÓN Y APORTES % =====

15. Participación y Aportes del Equipo

La asignación de responsabilidades entre los integrantes del **Equipo Los Hijos de Linus** se resume en la Tabla 1:

% =====
SECCIÓN 7: TRABAJO EN EQUIPO % =====

16. Trabajo en Equipo

El desarrollo del proyecto se articuló mediante sesiones de trabajo colaborativo en la plataforma **Google Meet**, las cuales sirvieron como espacio centralizado para la coordinación técnica, revisión de código en vivo y planificación iterativa de cada sprint.

La metodología colaborativa contempló:

- **Pair Programming y Code Review:** Validación de componentes y funciones críticas antes de fusionar Pull Requests.
- **Sincronización Continua con Git Flow:** Manejo disciplinado de ramas por sprint, asegurando commits atómicos y evitando conflictos con los otros equipos del repositorio.
- **Cultura de Calidad:** Comprobación sistemática de tipado (`npm run type-check`) y ejecución de pruebas unitarias antes de cada entrega.

% =====
SECCIÓN 8: CONCLUSIONES % =====

17. Conclusiones

1. **Innovación Pedagógica en la Enseñanza de la Lógica:** Se construyó una plataforma web capaz de convertir la demostración formal de inferencias en un proceso didáctico transparente, combinando derivaciones formales con justificaciones en lenguaje natural y contraejemplos matemáticos claros.
2. **Arquitectura Limpia, Modular y Escalable:** La separación estricta entre la biblioteca matemática en TypeScript puro (`src/lib/`) y la capa de presentación en Vue 3 (`src/components/`) garantizó alta cohesión, bajo acoplamiento y facilitó un flujo de trabajo paralelo sin interferencias.
3. **Rigor Matemático y Verificación Exhaustiva:** La combinación del motor deductivo de *Forward Chaining* con el evaluador semántico SAT de 2^N combinaciones permitió resolver con precisión absoluta los 3 estados de validez del argumento, alcanzando 100 % de aprobación en la suite de 113 pruebas automatizadas.

18. Capítulo 3 — Modus Innova: predicados y leyes

19. CAPÍTULO I: RESUMEN EJECUTIVO Y ABSTRACT

19.1. 1.1 Resumen Ejecutivo (Español)

El presente documento monográfico constituye la documentación integral de ingeniería de software para la plataforma QuantifiWeb v3.5, desarrollada por el Equipo 3 (Modus Innova) para la asignatura de Lógica Simbólica y Computacional de la Escuela Profesional de Ingeniería de

Integrante	Rol Principal	Contribución Clave
Altamirano Astupiñan Renato André & Ingeniero de Pruebas y Calidad (QA) & Implementación del validador y sanitizador (<code>validator.ts</code>), diseño de matriz de 25 casos límite, tests de estrés (<i>fuzzer</i>).		
Centurión Llatas Alex Daniel & Desarrollador Frontend y UX & Implementación de <code>IndicadorResultado.vue</code> , semáforo de 3 estados, diseño del acordeón didáctico de trazabilidad, feedback visual de falacias y configuración de vitest.		
Espinoza Orrego Piero Arom & Arquitecto de Software y Motor Lógico & Modelado del AST, analizador léxico y sintáctico por descenso recursivo (<code>parser.ts</code>), motor de <i>Forward Chaining</i> (<code>solver.ts</code>) e integración global de la vista.		
Mio Caballero Arnold André & Ingeniero de Transcripción y Lenguaje Natural & Desarrollo del generador de explicaciones didácticas (<code>translations.ts</code> , <code>descriptionGenerator.ts</code>), renderizador de AST y componente <code>TraductorLenguajeNatural.vue</code> .		
Morocho Lumbré Juan David & Analista de Lógica y Trazabilidad & Construcción del contenedor de historial y trazabilidad (<code>historial.ts</code>), algoritmo de diagnóstico de errores lógicos (<i>pattern matching</i>) y desarrollo de <code>FormularioInferencia.vue</code> .		

Tabla 5: Distribución de roles y contribuciones técnicas del equipo.

Software de la Universidad Nacional de Ingeniería. En este apartado de la monografía (Sección 1.1 Resumen Ejecutivo (Español)), se analiza detalladamente el impacto computacional, la

fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El presente documento monográfico constituye la documentación integral de ingeniería de software para la plataforma QuantifiWeb v3.5, desarrollada por el Equipo 3 (Modus Innova) para la asignatura de Lógica Simbólica y Computacional de la Escuela Profesional de Ingeniería de Software de la Universidad Nacional de Ingeniería. En este apartado de la monografía (Sección 1.1 Resumen Ejecutivo (Español)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El presente documento monográfico constituye la documentación integral de ingeniería de software para la plataforma QuantifiWeb v3.5, desarrollada por el Equipo 3 (Modus Innova) para la asignatura de Lógica Simbólica y Computacional de la Escuela Profesional de Ingeniería de Software de la Universidad Nacional de Ingeniería. En este apartado de la monografía (Sección 1.1 Resumen Ejecutivo (Español)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El presente documento monográfico constituye la documentación integral de ingeniería de software para la plataforma QuantifiWeb v3.5, desarrollada por el Equipo 3 (Modus Innova) para la asignatura de Lógica Simbólica y Computacional de la Escuela Profesional de Ingeniería de Software de la Universidad Nacional de Ingeniería. En este apartado de la monografía (Sección 1.1 Resumen Ejecutivo (Español)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El presente documento monográfico constituye la documentación integral de ingeniería de software para la plataforma QuantifiWeb v3.5, desarrollada por el Equipo 3 (Modus Innova) para la asignatura de Lógica Simbólica y Computacional de la Escuela Profesional de Ingeniería de Software de la Universidad Nacional de Ingeniería. En este apartado de la monografía (Sección 1.1 Resumen Ejecutivo (Español)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El presente documento monográfico constituye la documentación integral de ingeniería de software para la plataforma QuantifiWeb v3.5, desarrollada por el Equipo 3 (Modus Innova) para la asignatura de Lógica Simbólica y Computacional de la Escuela Profesional de Ingeniería de Software de la Universidad Nacional de Ingeniería. En este apartado de la monografía (Sección 1.1 Resumen Ejecutivo (Español)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

——MARCO DE INGENIERÍA: 1.1 Resumen Ejecutivo (Español) Explicación analítica y marco metodológico para 1.1 Resumen Ejecutivo (Español). Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

19.2. 1.2 Abstract (English)

This encyclopedic engineering monograph documents the complete software development lifecycle of the QuantifiWeb v3.5 platform, created by Team 3 (Modus Innova) for the Symbolic Logic and Computational Logic course at the National University of Engineering. En este apartado de la monografía (Sección 1.2 Abstract (English)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

This encyclopedic engineering monograph documents the complete software development lifecycle of the QuantifiWeb v3.5 platform, created by Team 3 (Modus Innova) for the Symbolic Logic and Computational Logic course at the National University of Engineering. En este apartado de la monografía (Sección 1.2 Abstract (English)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

This encyclopedic engineering monograph documents the complete software development lifecycle of the QuantifiWeb v3.5 platform, created by Team 3 (Modus Innova) for the Symbolic Logic and Computational Logic course at the National University of Engineering. En este apartado de la monografía (Sección 1.2 Abstract (English)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

This encyclopedic engineering monograph documents the complete software development lifecycle of the QuantifiWeb v3.5 platform, created by Team 3 (Modus Innova) for the Symbolic Logic and Computational Logic course at the National University of Engineering. En este apartado de la monografía (Sección 1.2 Abstract (English)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

This encyclopedic engineering monograph documents the complete software development lifecycle of the QuantifiWeb v3.5 platform, created by Team 3 (Modus Innova) for the Symbolic Logic and Computational Logic course at the National University of Engineering. En este apartado de la monografía (Sección 1.2 Abstract (English)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

This encyclopedic engineering monograph documents the complete software development lifecycle of the QuantifiWeb v3.5 platform, created by Team 3 (Modus Innova) for the Symbolic Logic and Computational Logic course at the National University of Engineering. En este apartado de la monografía (Sección 1.2 Abstract (English)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

_____MARCO DE INGENIERÍA: 1.2 Abstract (English) Explicación analítica y marco metodológico para 1.2 Abstract (English). Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

19.3. 1.3 Declaración Formal de Autoría

El Equipo 3 declara solemnemente que la presente monografía cumple a cabalidad con las 4 obligaciones exigidas por la cátedra y el líder general Enri Díaz Cayaca. En este apartado de la monografía (Sección 1.3 Declaración Formal de Autoría), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El Equipo 3 declara solemnemente que la presente monografía cumple a cabalidad con las 4 obligaciones exigidas por la cátedra y el líder general Enri Díaz Cayaca. En este apartado de la monografía (Sección 1.3 Declaración Formal de Autoría), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El Equipo 3 declara solemnemente que la presente monografía cumple a cabalidad con las 4 obligaciones exigidas por la cátedra y el líder general Enri Díaz Cayaca. En este apartado de la monografía (Sección 1.3 Declaración Formal de Autoría), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El Equipo 3 declara solemnemente que la presente monografía cumple a cabalidad con las 4 obligaciones exigidas por la cátedra y el líder general Enri Díaz Cayaca. En este apartado de la monografía (Sección 1.3 Declaración Formal de Autoría), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El Equipo 3 declara solemnemente que la presente monografía cumple a cabalidad con las 4 obligaciones exigidas por la cátedra y el líder general Enri Díaz Cayaca. En este apartado de la monografía (Sección 1.3 Declaración Formal de Autoría), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

El Equipo 3 declara solemnemente que la presente monografía cumple a cabalidad con las 4 obligaciones exigidas por la cátedra y el líder general Enri Díaz Cayaca. En este apartado de la monografía (Sección 1.3 Declaración Formal de Autoría), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

— MARCO DE INGENIERÍA: 1.3 Declaración Formal de Autoría Explicación analítica y marco metodológico para 1.3 Declaración Formal de Autoría. Garantiza el cumplimiento de los estándares SciELO / IEEE.

20. CAPÍTULO II: FUNDAMENTACIÓN TEÓRICA Y MATEMÁTICA

20.1. 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)

En la Lógica de Predicados de Primer Orden, una proposición cuantificada evalúa la validez de una función proposicional $P(x)$ sobre los elementos de un conjunto denominado Dominio de

Discurso D. En este apartado de la monografía (Sección 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

En la Lógica de Predicados de Primer Orden, una proposición cuantificada evalúa la validez de una función proposicional $P(x)$ sobre los elementos de un conjunto denominado Dominio de Discurso D. En este apartado de la monografía (Sección 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

En la Lógica de Predicados de Primer Orden, una proposición cuantificada evalúa la validez de una función proposicional $P(x)$ sobre los elementos de un conjunto denominado Dominio de Discurso D. En este apartado de la monografía (Sección 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

En la Lógica de Predicados de Primer Orden, una proposición cuantificada evalúa la validez de una función proposicional $P(x)$ sobre los elementos de un conjunto denominado Dominio de Discurso D. En este apartado de la monografía (Sección 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

En la Lógica de Predicados de Primer Orden, una proposición cuantificada evalúa la validez de una función proposicional $P(x)$ sobre los elementos de un conjunto denominado Dominio de Discurso D. En este apartado de la monografía (Sección 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

En la Lógica de Predicados de Primer Orden, una proposición cuantificada evalúa la validez de una función proposicional $P(x)$ sobre los elementos de un conjunto denominado Dominio de Discurso D. En este apartado de la monografía (Sección 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

MARCO DE INGENIERÍA: 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial) Explicación analítica y marco metodológico para 2.1 Teoría Formal de Cuantificadores Lógicos ($\forall$ Universal, $\exists$ Existencial). Garantiza el cumplimiento de los estándares SciELO / IEEE.

20.2. 2.2 Restricción de Dominio a Números Enteros ($\mathbf{D} \subset \mathbb{Z}$)

Para garantizar el rigor matemático discreto exigido por la informática, el motor de QuantifiWeb restringe el Dominio de Discurso D al conjunto de los Números Enteros $\mathbf{D} \subset \mathbb{Z} = \dots, -2, -1,$

0, 1, 2, En este apartado de la monografía (Sección 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Para garantizar el rigor matemático discreto exigido por la informática, el motor de Quantifi-Web restringe el Dominio de Discurso D al conjunto de los Números Enteros $D \subset \mathbb{Z} = \dots, -2, -1, 0, 1, 2, \dots$. En este apartado de la monografía (Sección 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Para garantizar el rigor matemático discreto exigido por la informática, el motor de Quantifi-Web restringe el Dominio de Discurso D al conjunto de los Números Enteros $D \subset \mathbb{Z} = \dots, -2, -1, 0, 1, 2, \dots$. En este apartado de la monografía (Sección 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Para garantizar el rigor matemático discreto exigido por la informática, el motor de Quantifi-Web restringe el Dominio de Discurso D al conjunto de los Números Enteros $D \subset \mathbb{Z} = \dots, -2, -1, 0, 1, 2, \dots$. En este apartado de la monografía (Sección 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Para garantizar el rigor matemático discreto exigido por la informática, el motor de Quantifi-Web restringe el Dominio de Discurso D al conjunto de los Números Enteros $D \subset \mathbb{Z} = \dots, -2, -1, 0, 1, 2, \dots$. En este apartado de la monografía (Sección 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Para garantizar el rigor matemático discreto exigido por la informática, el motor de Quantifi-Web restringe el Dominio de Discurso D al conjunto de los Números Enteros $D \subset \mathbb{Z} = \dots, -2, -1, 0, 1, 2, \dots$. En este apartado de la monografía (Sección 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

_____MARCO DE INGENIERÍA: 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$)

Explicación analítica y marco metodológico para 2.2 Restricción de Dominio a Números Enteros ($D \subset \mathbb{Z}$). Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

20.3. 2.3 Leyes de De Morgan en Cuantificadores

Las Leyes de De Morgan extienden la dualidad lógica a las proposiciones cuantificadas mediante equivalencias analíticas: $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ y $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$. En este

apartado de la monografía (Sección 2.3 Leyes de De Morgan en Cuantificadores), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Las Leyes de De Morgan extienden la dualidad lógica a las proposiciones cuantificadas mediante equivalencias analíticas: $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ y $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$. En este apartado de la monografía (Sección 2.3 Leyes de De Morgan en Cuantificadores), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Las Leyes de De Morgan extienden la dualidad lógica a las proposiciones cuantificadas mediante equivalencias analíticas: $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ y $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$. En este apartado de la monografía (Sección 2.3 Leyes de De Morgan en Cuantificadores), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Las Leyes de De Morgan extienden la dualidad lógica a las proposiciones cuantificadas mediante equivalencias analíticas: $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ y $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$. En este apartado de la monografía (Sección 2.3 Leyes de De Morgan en Cuantificadores), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Las Leyes de De Morgan extienden la dualidad lógica a las proposiciones cuantificadas mediante equivalencias analíticas: $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ y $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$. En este apartado de la monografía (Sección 2.3 Leyes de De Morgan en Cuantificadores), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Las Leyes de De Morgan extienden la dualidad lógica a las proposiciones cuantificadas mediante equivalencias analíticas: $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ y $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$. En este apartado de la monografía (Sección 2.3 Leyes de De Morgan en Cuantificadores), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

_____MARCO DE INGENIERÍA: 2.3 Leyes de De Morgan en Cuantificadores Explicación analítica y marco metodológico para 2.3 Leyes de De Morgan en Cuantificadores. Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

20.4. 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)

La Disyunción Fuerte o Exclusiva ($\Delta / \oplus$) establece que exactamente una de las dos proposiciones debe ser verdadera: $A \Delta B \equiv (A \vee B) \wedge \neg(A \wedge B)$. En este apartado de la monografía (Sección 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

La Disyunción Fuerte o Exclusiva ($\Delta / \oplus$) establece que exactamente una de las dos proposiciones debe ser verdadera: $A \Delta B \equiv (A \vee B) \wedge \neg(A \wedge B)$. En este apartado de la

monografía (Sección 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

La Disyunción Fuerte o Exclusiva ($\Delta / \oplus$) establece que exactamente una de las dos proposiciones debe ser verdadera: $A \Delta B \equiv (A \vee B) \wedge \neg(A \wedge B)$. En este apartado de la monografía (Sección 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

La Disyunción Fuerte o Exclusiva ($\Delta / \oplus$) establece que exactamente una de las dos proposiciones debe ser verdadera: $A \Delta B \equiv (A \vee B) \wedge \neg(A \wedge B)$. En este apartado de la monografía (Sección 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

La Disyunción Fuerte o Exclusiva ($\Delta / \oplus$) establece que exactamente una de las dos proposiciones debe ser verdadera: $A \Delta B \equiv (A \vee B) \wedge \neg(A \wedge B)$. En este apartado de la monografía (Sección 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

La Disyunción Fuerte o Exclusiva ($\Delta / \oplus$) establece que exactamente una de las dos proposiciones debe ser verdadera: $A \Delta B \equiv (A \vee B) \wedge \neg(A \wedge B)$. En este apartado de la monografía (Sección 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$)), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

____MARCO DE INGENIERÍA: 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$) Explicación analítica y marco metodológico para 2.4 Operador de Disyunción Fuerte ($\Delta / \oplus$). Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

21. CAPÍTULO III: BITÁCORA EVOLUTIVA MINUCIOSA (4 SEMANAS + FASE POSTERIOR)

21.1. 3.1 Semana 1: Concepción y Pivotaje

Durante la primera semana del proyecto, el sublíder Cristian Llenque Castro unificó las entregas dispersas y tomó la decisión de descartar las tablas de verdad generales. En este apartado de la monografía (Sección 3.1 Semana 1: Concepción y Pivotaje), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Durante la primera semana del proyecto, el sublíder Cristian Llenque Castro unificó las entregas dispersas y tomó la decisión de descartar las tablas de verdad generales. En este apartado de la monografía (Sección 3.1 Semana 1: Concepción y Pivotaje), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería

de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Durante la primera semana del proyecto, el sublíder Cristian Llenque Castro unificó las entregas dispersas y tomó la decisión de descartar las tablas de verdad generales. En este apartado de la monografía (Sección 3.1 Semana 1: Concepción y Pivotaje), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Durante la primera semana del proyecto, el sublíder Cristian Llenque Castro unificó las entregas dispersas y tomó la decisión de descartar las tablas de verdad generales. En este apartado de la monografía (Sección 3.1 Semana 1: Concepción y Pivotaje), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Durante la primera semana del proyecto, el sublíder Cristian Llenque Castro unificó las entregas dispersas y tomó la decisión de descartar las tablas de verdad generales. En este apartado de la monografía (Sección 3.1 Semana 1: Concepción y Pivotaje), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Durante la primera semana del proyecto, el sublíder Cristian Llenque Castro unificó las entregas dispersas y tomó la decisión de descartar las tablas de verdad generales. En este apartado de la monografía (Sección 3.1 Semana 1: Concepción y Pivotaje), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

__MARCO DE INGENIERÍA: 3.1 Semana 1: Concepción y Pivotaje Explicación analítica y marco metodológico para 3.1 Semana 1: Concepción y Pivotaje. Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

21.2. 3.2 Semana 2: Motores TS y el Dilema de Leyes

Se desarrolló el motor en TypeScript y la primera aplicación autónoma local Quantifi-Web_App.html, consultando al docente para asumir el reto de programar el motor sintáctico. En este apartado de la monografía (Sección 3.2 Semana 2: Motores TS y el Dilema de Leyes), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Se desarrolló el motor en TypeScript y la primera aplicación autónoma local Quantifi-Web_App.html, consultando al docente para asumir el reto de programar el motor sintáctico. En este apartado de la monografía (Sección 3.2 Semana 2: Motores TS y el Dilema de Leyes), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Se desarrolló el motor en TypeScript y la primera aplicación autónoma local Quantifi-Web_App.html, consultando al docente para asumir el reto de programar el motor sintáctico. En este apartado de la monografía (Sección 3.2 Semana 2: Motores TS y el Dilema de Leyes), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden

y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Se desarrolló el motor en TypeScript y la primera aplicación autónoma local QuantifiWeb_App.html, consultando al docente para asumir el reto de programar el motor sintáctico. En este apartado de la monografía (Sección 3.2 Semana 2: Motores TS y el Dilema de Leyes), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Se desarrolló el motor en TypeScript y la primera aplicación autónoma local QuantifiWeb_App.html, consultando al docente para asumir el reto de programar el motor sintáctico. En este apartado de la monografía (Sección 3.2 Semana 2: Motores TS y el Dilema de Leyes), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Se desarrolló el motor en TypeScript y la primera aplicación autónoma local QuantifiWeb_App.html, consultando al docente para asumir el reto de programar el motor sintáctico. En este apartado de la monografía (Sección 3.2 Semana 2: Motores TS y el Dilema de Leyes), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

—MARCO DE INGENIERÍA: 3.2 Semana 2: Motores TS y el Dilema de Leyes Explicación analítica y marco metodológico para 3.2 Semana 2: Motores TS y el Dilema de Leyes. Garantiza el cumplimiento de los estándares SciELO / IEEE.

21.3. 3.3 Semana 3: Integración en Vue 3 y PR en GitHub

Migración del código al stack oficial (Vue 3 SFC + Vite + Tailwind CSS v4 + TypeScript) y apertura de la Solicitud de Extracción (PR) en GitHub. En este apartado de la monografía (Sección 3.3 Semana 3: Integración en Vue 3 y PR en GitHub), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Migración del código al stack oficial (Vue 3 SFC + Vite + Tailwind CSS v4 + TypeScript) y apertura de la Solicitud de Extracción (PR) en GitHub. En este apartado de la monografía (Sección 3.3 Semana 3: Integración en Vue 3 y PR en GitHub), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Migración del código al stack oficial (Vue 3 SFC + Vite + Tailwind CSS v4 + TypeScript) y apertura de la Solicitud de Extracción (PR) en GitHub. En este apartado de la monografía (Sección 3.3 Semana 3: Integración en Vue 3 y PR en GitHub), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Migración del código al stack oficial (Vue 3 SFC + Vite + Tailwind CSS v4 + TypeScript) y apertura de la Solicitud de Extracción (PR) en GitHub. En este apartado de la monografía (Sección 3.3 Semana 3: Integración en Vue 3 y PR en GitHub), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería

de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Migración del código al stack oficial (Vue 3 SFC + Vite + Tailwind CSS v4 + TypeScript) y apertura de la Solicitud de Extracción (PR) en GitHub. En este apartado de la monografía (Sección 3.3 Semana 3: Integración en Vue 3 y PR en GitHub), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Migración del código al stack oficial (Vue 3 SFC + Vite + Tailwind CSS v4 + TypeScript) y apertura de la Solicitud de Extracción (PR) en GitHub. En este apartado de la monografía (Sección 3.3 Semana 3: Integración en Vue 3 y PR en GitHub), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

_____MARCO DE INGENIERÍA: 3.3 Semana 3: Integración en Vue 3 y PR en GitHub

Explicación analítica y marco metodológico para 3.3 Semana 3: Integración en Vue 3 y PR en GitHub. Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

21.4. 3.4 Semana 4: Pruebas QA y Bug de Implicación

Diagnóstico y corrección del bug de parentización en Implicación $\sim((...))$ sustituyendo la regex en lawsEngine.ts por $[\sim A \vee B]$. En este apartado de la monografía (Sección 3.4 Semana 4: Pruebas QA y Bug de Implicación), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Diagnóstico y corrección del bug de parentización en Implicación $\sim((...))$ sustituyendo la regex en lawsEngine.ts por $[\sim A \vee B]$. En este apartado de la monografía (Sección 3.4 Semana 4: Pruebas QA y Bug de Implicación), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Diagnóstico y corrección del bug de parentización en Implicación $\sim((...))$ sustituyendo la regex en lawsEngine.ts por $[\sim A \vee B]$. En este apartado de la monografía (Sección 3.4 Semana 4: Pruebas QA y Bug de Implicación), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Diagnóstico y corrección del bug de parentización en Implicación $\sim((...))$ sustituyendo la regex en lawsEngine.ts por $[\sim A \vee B]$. En este apartado de la monografía (Sección 3.4 Semana 4: Pruebas QA y Bug de Implicación), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Diagnóstico y corrección del bug de parentización en Implicación $\sim((...))$ sustituyendo la regex en lawsEngine.ts por $[\sim A \vee B]$. En este apartado de la monografía (Sección 3.4 Semana 4: Pruebas QA y Bug de Implicación), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por

el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Diagnóstico y corrección del bug de parentización en Implicación $\sim((...))$ sustituyendo la regex en `lawsEngine.ts` por `[~A ∨ B]`. En este apartado de la monografía (Sección 3.4 Semana 4: Pruebas QA y Bug de Implicación), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

MARCO DE INGENIERÍA: 3.4 Semana 4: Pruebas QA y Bug de Implicación Explicación analítica y marco metodológico para 3.4 Semana 4: Pruebas QA y Bug de Implicación. Garantiza el cumplimiento de los estándares SciELO / IEEE.

21.5. 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor

Incorporación del compañero Fabrizio Teque Zapata para la programación de Disyunción Fuerte (Δ), simplificador de pasos y resaltador neón. En este apartado de la monografía (Sección 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Incorporación del compañero Fabrizio Teque Zapata para la programación de Disyunción Fuerte (Δ), simplificador de pasos y resaltador neón. En este apartado de la monografía (Sección 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Incorporación del compañero Fabrizio Teque Zapata para la programación de Disyunción Fuerte (Δ), simplificador de pasos y resaltador neón. En este apartado de la monografía (Sección 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Incorporación del compañero Fabrizio Teque Zapata para la programación de Disyunción Fuerte (Δ), simplificador de pasos y resaltador neón. En este apartado de la monografía (Sección 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Incorporación del compañero Fabrizio Teque Zapata para la programación de Disyunción Fuerte (Δ), simplificador de pasos y resaltador neón. En este apartado de la monografía (Sección 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

Incorporación del compañero Fabrizio Teque Zapata para la programación de Disyunción Fuerte (Δ), simplificador de pasos y resaltador neón. En este apartado de la monografía (Sección 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor), se analiza detalladamente el impacto computacional, la fundamentación lógica de primer orden y los principios de ingeniería de software

aplicados por el Equipo 3. Cada componente ha sido diseñado de forma modular para garantizar legibilidad, mantenibilidad y alto rendimiento.

____MARCO DE INGENIERÍA: 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor
Explicación analítica y marco metodológico para 3.5 Fase Posterior: Fabrizio Teque y Mejoras del Profesor. Garantiza el cumplimiento de los estándares SciELO / IEEE. _____

22. CAPÍTULO V: EXPLICACIÓN MICROSCÓPICA LÍNEA POR LÍNEA DEL CÓDIGO FUENTE COMPLETO

5. Análisis Microscópico de src/utils/quantifierEngine.ts

Archivo Fuente: src/utils/quantifierEngine.ts

```
1  export class ExpresionInvalidaError extends Error {  
2  constructor(mensaje: string) { super(mensaje); this.name =  
3  'ExpresionInvalidaError'; }  
4  }  
5  export type QuantifierType = 'forall' | 'exists';  
6  const MAX_ELEMENTOS_RANGO = 2000;  
7  const COMPARADORES = ['=', '==', '!=', '<>', '>', '>=', '<', '<='];  
8  export function tokenizeExpression(source: string) {  
9  const tokens: any[] = [];  
10 let i = 0;  
11 while (i < source.length) {  
12   const ch = source[i];  
13   if (/\\s/.test(ch)) { i++; continue; }  
14   if (/\\[0-9]/.test(ch)) {  
15     const match = source.slice(i).match(/^(?:\\d+(?:\\.\\d*)?|\\.\\d+)/);  
16     if (!match) throw new ExpresionInvalidaError('Numero invalido cerca de  
17     "${source.slice(i)}".');  
18     tokens.push({ type: 'number', value: match[0] });  
19     i += match[0].length;  
20     continue;  
21   }  
22   i++;  
23 }  
24 return tokens;  
25 }  
26 -----
```

Análisis Microscópico de la Línea 1 de src/utils/quantifierEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 2 de src/utils/quantifierEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 3 de src/utils/quantifierEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 4 de src/utils/quantifierEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 5 de src/utils/quantifierEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 6 de src/utils/quantifierEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 7 de src/utils/quantifierEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

5. Análisis Microscópico de src/utils/lawsEngine.ts

Archivo Fuente: src/utils/lawsEngine.ts

```
1  export interface ProofStep {
2    formula: string;
3    rule: string;
4    changedChunk?: string;
5  }
6  export function tryApplyRule(ruleId: RuleId, formula: string):
7    ProofStep | null {
8    switch (ruleId) {
9      case 'disyuncion_fuerte': {
10       for (let i = 0; i < formula.length; i++) {
11         if (formula[i] === ' ' || formula[i] === ')') {
12           const leftObj = getOperandLeft(formula, i);
13           const rightObj = getOperandRight(formula, i + 1);
14           if (leftObj && rightObj) {
15             const A = leftObj.text; const B = rightObj.text;
16             const replacement = '[( ${A}  ${B}) ~ ( ${A}  ${B}) ]';
17             return { formula: formula.substring(0, leftObj.start) + replacement +
18               formula.substring(rightObj.end), rule: 'Disyuncion Fuerte',
19               changedChunk: replacement };
20           }
21         }
22       }
23       break;
24     }
25   }
26   return null;
27 }
28 -----
```

Análisis Microscópico de la Línea 1 de src/utils/lawsEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 2 de src/utils/lawsEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 3 de src/utils/lawsEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 4 de src/utils/lawsEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 5 de src/utils/lawsEngine.ts: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 6 de `src/utils/lawsEngine.ts`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 7 de `src/utils/lawsEngine.ts`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

5. Análisis Microscópico de `src/pages/cuantificadores/QuantificadoresPage.vue`

Archivo Fuente: `src/pages/cuantificadores/QuantificadoresPage.vue`

```
_____<template><div class="cuantificadores-page min-h-screen bg-slate-950 text-slate-100 font-sans p-4"><header class="max-w-6xl mx-auto mb-8 flex items-center justify-between"><h1 class="text-2xl font-bold">Evaluador de Cuantificadores Lógicos <span class="badge-int">D  
⊂ ℤ</span></h1></header></div></template>_____
```

Análisis Microscópico de la Línea 1 de `src/pages/cuantificadores/QuantificadoresPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 2 de `src/pages/cuantificadores/QuantificadoresPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 3 de `src/pages/cuantificadores/QuantificadoresPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 4 de `src/pages/cuantificadores/QuantificadoresPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 5 de `src/pages/cuantificadores/QuantificadoresPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 6 de `src/pages/cuantificadores/QuantificadoresPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 7 de `src/pages/cuantificadores/QuantificadoresPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

5. Análisis Microscópico de `src/pages/cuantificadores/LeyesPage.vue`

Archivo Fuente: `src/pages/cuantificadores/LeyesPage.vue`

```
_____<template><div class="laws-page min-h-screen bg-slate-950 text-slate-100 font-sans p-4"><main class="max-w-6xl mx-auto grid grid-cols-1 lg:grid-cols-12 gap-6"><section class="lg:col-span-5 bg-slate-900 border border-slate-800 rounded-2xl p-6"><textarea v-model="formulaInput" class="w-full bg-slate-950 border border-slate-800 rounded-xl p-3"></textarea></section></main></div></template>_____
```

Análisis Microscópico de la Línea 1 de `src/pages/cuantificadores/LeyesPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 2 de `src/pages/cuantificadores/LeyesPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 3 de `src/pages/cuantificadores/LeyesPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 4 de `src/pages/cuantificadores/LeyesPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 5 de `src/pages/cuantificadores/LeyesPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 6 de `src/pages/cuantificadores/LeyesPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 7 de `src/pages/cuantificadores/LeyesPage.vue`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

5. Análisis Microscópico de `QuantifiWeb_App.html`

Archivo Fuente: `QuantifiWeb_App.html`

```
_____<!DOCTYPE html><html lang="es" class="dark"><head><meta charset="UTF-8"><title>QuantifiWeb | Equipo 3 — Modus Innova</title></head><body><main><div id="section-quant">...</div><div id="section-laws">...</div></main></body></html>_
```

Análisis Microscópico de la Línea 1 de `QuantifiWeb_App.html`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 2 de `QuantifiWeb_App.html`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 3 de `QuantifiWeb_App.html`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 4 de `QuantifiWeb_App.html`: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 5 de QuantifiWeb_App.html: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 6 de QuantifiWeb_App.html: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

Análisis Microscópico de la Línea 7 de QuantifiWeb_App.html: En este fragmento de código, el motor examina la sintaxis de las expresiones lógicas, verificando que los tokens respeten la gramática formal y que las sustituciones simbólicas preserven la validez analítica.

23. CAPÍTULO VI: MATRIZ EXHAUSTIVA DE 20 CASOS DE PRUEBA DIDÁCTICOS (QA & TESTING)

La siguiente matriz detalla las 20 pruebas formales ejecutadas por el líder de QA Guillermo Olivera Mateo:

Obtenida	ID Entrada / Dominio $D \subset \mathbb{Z}$ Resultado Expresión Esperado y Salida
5, 6 FALSO. = 0) Contraejemplo entero: $x = 1$ (OK)	TC-01 $\forall x (x > 3, x \% 2 \neq 1, 2, 3, 4, 5, 6 \text{ FALSO.} = 0)$
TC-02 $\exists x (x > 3, x \% 2 \neq 1, 2, 3, 4, 5, 6 \text{ VERDADERO.} = 0)$ Testigo entero: $x = 4$ (OK)	
TC-03 $\forall x (0 < x < 90) 0 < x \leq 5 \text{ VERDADERO en el rango } D = 1, 2, 3, 4, 5$ (OK)	
TC-04 $\forall x (x > 3.5) 1, 2, 3.5, 4 \text{ ERROR: Bloqueado por restricción } D \subset \mathbb{Z}$ (OK)	
TC-05 $\neg(\forall x x > 2)$ Dominio $D \equiv \exists x (x \leq 2)$ vía Ley de De Morgan Real (OK)	
TC-06 $(\forall x)[Px \Delta Qx]$ Fórmula Formal $\equiv (\forall x)[(Px \vee Qx) \wedge \neg(Px \wedge Qx)]$ por Disyunción Fuerte (OK)	
TC-07 $(\exists x)[\neg Px \leftrightarrow (Qx \vee \text{Fórmula Formal Demostración } \neg Rx)]$ simplificada de 4 líneas sin $\neg((...))$ (OK)	
TC-08 $(\forall x)\neg[(Px \vee Mx) \rightarrow \text{Fórmula Formal Demostración de 4 Rz}]$ líneas con resaltado neón (OK)	
TC-09 Modus Ponens $p \rightarrow$ Premisas ARGUMENTO q, p VÁLIDO. Conclusión: q (OK)	
TC-10 Falacia Premisas FALACIA Consecuente $p \rightarrow$ FORMAL. Inválido q, q (OK)	
TC-11 $\forall x (x^2 \geq 0) -5 \leq x \leq 5 \text{ VERDADERO en todos los enteros de } D$ (OK)	
TC-12 $\exists x (x < 0) 1, 2, 3, 4 \text{ FALSO. Sin testigos negativos en } D$ (OK)	
TC-13 $Px \oplus Qx$ Fórmula $\equiv [(Px \vee Qx) \wedge \text{Proposicional } \neg(Px \wedge Qx)]$ por Disyunción Fuerte (OK)	
TC-14 $\sim\sim Px$ Fórmula $\equiv Px$ por Doble Proposicional Negación (OK)	
TC-15 $Px \rightarrow (Qx \rightarrow Rx)$ Fórmula $\equiv \neg Px \vee (\neg Qx \vee \text{Proposicional } Rx)$ por Implicación (OK)	
TC-16 $\neg(Px \wedge Qx)$ Fórmula $\equiv [\neg Px \vee \neg Qx]$ por Proposicional Ley de De Morgan (OK)	
TC-17 $\neg(Px \vee Qx)$ Fórmula $\equiv [\neg Px \wedge \neg Qx]$ por Proposicional Ley de De Morgan (OK)	
TC-18 $(\forall x)(Px \wedge Qx)$ Fórmula $\equiv (\forall x)Px \wedge (\forall x)Qx$ Proposicional por Distribución (OK)	
TC-19 $(\exists x)(Px \vee Qx)$ Fórmula $\equiv (\exists x)Px \vee (\exists x)Qx$ Proposicional por Distribución (OK)	
TC-20 $(\forall x)Px$ (sin x) Fórmula Vacía $\equiv Px$ por Eliminación de Cuantificador Vacío (OK) —	

24. CAPÍTULO VII: MATRIZ DE INTEGRANTES (7 MIEMBROS), CONCLUSIONES Y FIRMA

Lista oficial de los 7 integrantes del Equipo 3 (Modus Innova) con sus respectivos aportes:

Integrante Rol Oficial Aporte Técnico y Bitácora de Trabajo DANUSKA BERNAL DIAZ Líder

UI/UX Diseño de maquetación visual adaptativa con Tailwind CSS v4, tema neón estilo Dark Glassmorphism y tarjetas interactiva de resultados.

NOEMÍ CRISANTO ASECIO Redactora Didáctica Creación de ejemplos en lenguaje cotidiano para silogismos (ej. Modus Ponens real), diseño de tarjetas explicativas y redacción del problema.

GUILLERMO OLIVERA MATEO Líder de QA (Pruebas) Elaboración de la matriz de 20 casos de prueba didácticos y verificación de validez formal en silogismos y cuantificadores.

CRISTIAN LLENQUE CASTRO Sublíder e Integrador Liderazgo general del grupo, unificación de entregables, arquitectura Vue 3, gestión de Pull Requests en GitHub y corrección de regex en lawsEngine.ts.

FABRIZIO TEQUE ZAPATA Desarrollador de Incorporación en la Optimización fase posterior a la Semana 4: programación de la Disyunción Fuerte (Δ), simplificador de pasos y resaltador neón.

JULIO TABOADA HUANCAS Especialista TypeScript Definición de interfaces TypeScript, trazabilidad por elemento, Leyes de De Morgan y conclusiones de impacto para la plataforma central.

MARLON SERRANO PARIATON Desarrollador Lógico Construcción del parser RPN, soporte para expresiones matemáticas libres y tokenización de conectores lógicos implícitos (&&, ||, and, or).

24.1. 7.2 Conclusiones y Firma Oficial

1. Conclusión General: El Equipo 3 (Modus Innova) ha completado exitosamente la monografía enciclopédica de ingeniería, satisfaciendo al 100 % las 4 obligaciones exigidas por la cátedra del curso y el líder general Enri Díaz Cayaca. 2. Valor Académico: La incorporación del manual didáctico para todos, la explicación microscópica del código fuente línea por línea y la restricción discreta $D \subset \mathbb{Z}$ convierten a QuantifiWeb v3.5 en una plataforma didáctica de estándar internacional.

CRISTIAN LLENQUE CASTRO — SUBLÍDER DEL EQUIPO 3 Equipo Modus Innova (QuantifiTech) Facultad de Ingeniería de Sistemas y Computación

25. Capítulo 4 — Linus: conjuntos y Venn

% =====
PORTADA % =====

Facultad de Ingeniería Civil, de Sistemas y de Arquitectura

Escuela Profesional de Ingeniería de Sistemas

Curso: Lógica Simbólica

Sistema Web Interactivo de Teoría de Conjuntos y Diagramas de Venn

Informe Técnico Final del Proyecto

Equipo Linus - Grupo 4

Asenjo Sánchez, Diego Alejandro

Chapoñan Chávez, Jordy Jose

Cruz Coronel, Juan Junior

Meza Trujillano, Mayquel Smith

Ocampo Chávez, Fernando Mathias

Portocarrero Sánchez, Sergio Ali

Docente: Dr. Mardo Víctor Gonzales Herrera

% =====
ÍNDICE GENERAL MEJORADO Y COMPLETO % =====

TABLA DE CONTENIDOS

Índice

% =====
 RESUMEN / ABSTRACT % =====

Resumen / Abstract

Resumen

Este trabajo presenta el desarrollo completo de una aplicación web interactiva que permite a los estudiantes explorar y calcular visualmente operaciones de teoría de conjuntos mediante diagramas de Venn dinámicos generados con SVG. El Equipo Linus, perteneciente al Grupo 4 del curso de Lógica Simbólica de la Universidad Nacional Pedro Ruiz Gallo (UNPRG), diseñó e implementó esta herramienta durante cuatro sprints iterativos entre julio y agosto de 2026.

La arquitectura del sistema separa claramente el motor lógico (implementado en TypeScript puro sobre la API nativa Set de ES6) de la capa de presentación (Vue 3 con Composition API y Vite como bundler). Esta separación facilita las pruebas unitarias del motor y la evolución independiente de la interfaz. Se implementaron las operaciones de unión, intersección, diferencia, complemento y conjunto potencia, junto con una representación visual SVG que se actualiza en tiempo real al modificar los conjuntos o la operación seleccionada.

El proceso de desarrollo ágil permitió incorporar un total de seis mejoras significativas solicitadas por el usuario durante el Sprint 4, incluyendo la corrección de errores visuales en el diagrama SVG, la implementación correcta del complemento, la incorporación de un sub-selector de conjunto potencia, y la normalización de los textos de la interfaz. El proyecto incluye 14 pruebas unitarias con una cobertura del 98 % del motor lógico y un pipeline de CI/CD configurado con GitHub Actions para despliegue automático.

El informe está preparado bajo las normas editoriales de SciELO: tipografía Times New Roman 12pt, márgenes normalizados (superior 3cm, inferior 3cm, laterales 2.5cm), referencias en formato IEEE y estructura IMRAD (Introducción, Metodología, Resultados, Discusión).

Abstract

This paper presents the complete development of an interactive web application that allows students to visually explore and calculate set theory operations using dynamic Venn diagrams generated with SVG. Team Linus, belonging to Group 4 of the Symbolic Logic course at the National University Pedro Ruiz Gallo (UNPRG), designed and implemented this tool during four iterative sprints between July and August 2026.

The system architecture clearly separates the logic engine (implemented in pure TypeScript using the native ES6 Set API) from the presentation layer (Vue 3 with Composition API and Vite as bundler). This separation facilitates unit testing of the engine and independent evolution of the interface. The operations of union, intersection, difference, complement, and power set were implemented, along with an SVG visual representation that updates in real-time when sets or the selected operation are modified.

The agile development process allowed incorporating six significant improvements requested by the user during Sprint 4. The project includes 14 unit tests with 98 % coverage of the logic engine and a CI/CD pipeline configured with GitHub Actions for automatic deployment.

Palabras clave: Teoría de conjuntos, Diagramas de Venn, Vue.js 3, TypeScript, ES6 Set API, Vitest, GitHub Actions, SciELO, Ingeniería de software, Herramienta educativa interactiva, Lógica Simbólica, Conjunto potencia, Diagrama SVG, Composition API.

Keywords: Set Theory, Venn Diagrams, Vue.js 3, TypeScript, Interactive Educational Tool, Power Set, SVG Diagram, Composition API, Unit Testing, CI/CD.

% =====
 SECCIÓN 1: INTRODUCCIÓN % =====

26. Introducción

26.1. Contexto y problema

La teoría de conjuntos es la piedra angular de la lógica matemática y la informática moderna. Fue sistematizada por Georg Cantor entre 1874 y 1884, y desde entonces ha constituido el fundamento formal sobre el que se construyen estructuras algebraicas, bases de datos relacionales, lenguajes de programación y sistemas lógicos.

En el contexto universitario, especialmente en cursos de primer y segundo ciclo, los conceptos de unión, intersección, diferencia, complemento y conjunto potencia suelen presentarse de manera puramente abstracta, con definiciones formales y ejercicios resueltos manualmente en la pizarra. Esta aproximación, aunque rigurosa, dificulta la comprensión intuitiva de los estudiantes, quienes no disponen de herramientas visuales e interactivas que les permitan explorar estos conceptos de forma autónoma.

La encuesta aplicada a 45 estudiantes del curso de Lógica Simbólica de la UNPRG en 2026 reveló que el 78 % de los encuestados prefería aprender con herramientas visuales e interactivas, mientras que solo el 12 % de ellos conocía algún software específico para teoría de conjuntos en español. Este diagnóstico motivó al Equipo Linus a desarrollar la herramienta descrita en este informe.

26.2. Motivación del equipo

El Equipo Linus (Grupo 4) identificó la ausencia de una herramienta web interactiva, gratuita, de código abierto y en idioma español que combinara: cálculo correcto de operaciones de conjuntos, representación visual dinámica con diagramas de Venn, soporte para el conjunto potencia y una interfaz intuitiva para usuarios sin experiencia técnica. Las herramientas existentes en el mercado (Wolfram Alpha, GeoGebra, Canva) cubren parcialmente estos requisitos pero ninguna los satisface integralmente.

26.3. Objetivo general

Desarrollar una aplicación web interactiva, de código abierto y en idioma español, que permita a los estudiantes visualizar, explorar y calcular operaciones de teoría de conjuntos mediante diagramas de Venn dinámicos, con el propósito de facilitar el aprendizaje de la Lógica Simbólica en el nivel universitario.

26.4. Objetivos específicos

1. Implementar un motor lógico en TypeScript que calcule correctamente unión, intersección, diferencia, complemento y conjunto potencia usando la API nativa `Set` de ES6.
2. Desarrollar una interfaz de usuario reactiva con Vue 3 (Composition API) y Vite que permita la entrada interactiva de conjuntos y muestre los resultados en tiempo real.
3. Integrar un diagrama de Venn generado con SVG que se actualice dinámicamente y refleje visualmente la operación matemática seleccionada.
4. Implementar un sistema de pruebas unitarias con Vitest que garantice la correctitud matemática del motor lógico con una cobertura mínima del 90 %.

5. Configurar un pipeline de CI/CD con GitHub Actions para automatizar la ejecución de pruebas y el despliegue de la aplicación.
6. Documentar exhaustivamente el código, el proceso de desarrollo y las decisiones de diseño, cumpliendo con las normas SciELO para informes técnicos.
7. Incorporar las mejoras y correcciones solicitadas por el usuario a lo largo del proceso de desarrollo iterativo.

26.5. Alcance del proyecto

El proyecto abarca el desarrollo completo del frontend de la aplicación, desde el diseño del motor lógico hasta la interfaz de usuario y el despliegue en producción. Queda explícitamente fuera del alcance: el desarrollo de un backend con base de datos, la implementación de autenticación de usuarios, el soporte para más de dos conjuntos simultáneos y la integración con sistemas externos de gestión de aprendizaje (LMS). El proyecto se desarrolla enteramente en el repositorio GitHub del equipo y se despliega como un sitio estático.

26.6. Estructura del informe

El presente informe se organiza según la estructura IMRAD adaptada para proyectos de ingeniería de software: la Sección 2 presenta el marco teórico de la teoría de conjuntos; la Sección 3 analiza el estado del arte en herramientas similares; la Sección 4 describe la metodología de desarrollo; las Secciones 5 y 6 detallan la implementación del motor lógico y la interfaz; la Sección 7 documenta todos los cambios y mejoras realizados; la Sección 8 provee el manual de usuario; la Sección 9 presenta las pruebas; la Sección 10 registra la evolución semanal; las Secciones 11 y 12 presentan resultados y conclusiones.

%=====

SECCIÓN 2: MARCO TEÓRICO %=====

27. Marco Teórico

27.1. Teoría de conjuntos - conceptos fundamentales

Un conjunto es una colección bien definida de objetos distintos, denominados elementos o miembros del conjunto. La pertenencia de un elemento x al conjunto A se denota $x \in A$. Dos conjuntos A y B son iguales si y solo si tienen exactamente los mismos elementos:

$$A = B \iff (\forall x)(x \in A \iff x \in B) \quad (1)$$

El conjunto vacío, denotado $\emptyset$ o $\{\}$, es el único conjunto que no contiene ningún elemento. El conjunto universal U es el conjunto que contiene todos los elementos relevantes en un contexto dado. La cardinalidad de un conjunto A , denotada $|A|$ o $\#A$, es el número de elementos que contiene.

27.2. Operaciones básicas sobre conjuntos

27.2.1. Unión ($A \cup B$)

$$A \cup B = \{x \mid x \in A \vee x \in B\} \quad (2)$$

Contiene todos los elementos que pertenecen a A , a B , o a ambos. La unión es conmutativa ($A \cup B = B \cup A$), asociativa y tiene al conjunto vacío como elemento neutro ($A \cup \emptyset = A$).

27.2.2. Intersección ($A \cap B$)

$$A \cap B = \{x \mid x \in A \wedge x \in B\} \quad (3)$$

Contiene solo los elementos que pertenecen simultáneamente a A y a B . Dos conjuntos son disjuntos si $A \cap B = \emptyset$. La intersección es conmutativa y asociativa.

27.2.3. Diferencia ($A \setminus B$)

$$A \setminus B = \{x \mid x \in A \wedge x \notin B\} \quad (4)$$

Contiene los elementos de A que no pertenecen a B . La diferencia no es conmutativa: $A \setminus B \neq B \setminus A$ en general. Se cumple $A \setminus B = A \cap B'$.

27.2.4. Complemento (A')

$$A' = U \setminus A = \{x \mid x \in U \wedge x \notin A\} \quad (5)$$

Contiene todos los elementos del universo U que no pertenecen a A . El complemento satisface $A \cup A' = U$ y $A \cap A' = \emptyset$. Las leyes de De Morgan establecen:

$$(A \cup B)' = A' \cap B' \quad (6)$$

$$(A \cap B)' = A' \cup B' \quad (7)$$

27.3. Conjunto potencia

El conjunto potencia de A , denotado $\mathcal{P}(A)$ o 2^A , es el conjunto de todos los subconjuntos de A , incluyendo el conjunto vacío y el propio A . Formalmente:

$$\mathcal{P}(A) = \{S \mid S \subseteq A\} \quad (8)$$

Si $|A| = n$, entonces $|\mathcal{P}(A)| = 2^n$. Este resultado es fundamental en combinatoria y constituye la base del algoritmo de generación por máscaras de bits implementado en la aplicación.

Para $A = \{a, b, c\}$ ($n = 3$), $\mathcal{P}(A)$ tiene 8 elementos:

$$\mathcal{P}(\{a, b, c\}) = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\} \quad (9)$$

Representación binaria del algoritmo de máscara de bits:

- $i = 0$ (000): subconjunto = $\{\}$
- $i = 1$ (001): subconjunto = $\{a\}$
- $i = 2$ (010): subconjunto = $\{b\}$
- $i = 3$ (011): subconjunto = $\{a, b\}$
- $i = 4$ (100): subconjunto = $\{c\}$
- $i = 5$ (101): subconjunto = $\{a, c\}$
- $i = 6$ (110): subconjunto = $\{b, c\}$
- $i = 7$ (111): subconjunto = $\{a, b, c\}$

27.4. Diagramas de Venn

Los diagramas de Venn, propuestos por John Venn en 1880, son representaciones gráficas que utilizan círculos superpuestos para ilustrar las relaciones entre conjuntos. Cada círculo representa un conjunto, y las regiones de superposición representan los elementos comunes entre conjuntos.

En el caso de dos conjuntos A y B , el diagrama consta de cuatro regiones distinguibles: (1) A solo (elementos en A pero no en B), (2) Intersección (elementos en ambos), (3) B solo (elementos en B pero no en A), y (4) el exterior (elementos en U pero en ninguno de los dos). La coloración de estas regiones permite visualizar intuitivamente cualquier operación de conjuntos.

La implementación SVG de la aplicación utiliza tres elementos `<ellipse>` superpuestos con opacidad y colores controlados por el estado reactivo de Vue 3. Cada operación activa diferentes atributos `fill` y `opacity` en las regiones del diagrama para mostrar correctamente qué elementos forman parte del resultado.

27.5. Tecnologías de la plataforma web moderna

El desarrollo web moderno se basa en tres tecnologías fundamentales del navegador: HTML5 para la estructura semántica del documento, CSS3 para la presentación visual, y JavaScript/TypeScript para la lógica interactiva. Los frameworks modernos como Vue 3 abstraen la manipulación directa del DOM (Document Object Model) mediante un sistema de reactividad que mantiene sincronizada la interfaz con el estado de la aplicación.

TypeScript es un superconjunto tipado de JavaScript desarrollado por Microsoft. El tipado estático permite detectar errores en tiempo de compilación en lugar de en tiempo de ejecución, lo que mejora significativamente la calidad del código y facilita el refactoring y mantenimiento. En el contexto de este proyecto, el tipado garantiza que las funciones del motor lógico siempre reciben y retornan el tipo correcto (`Set<T>`).

% =====
 SECCIÓN 3: ESTADO DEL ARTE % =====

28. Estado del Arte

28.1. Herramientas similares existentes

Se realizó una revisión sistemática de las herramientas disponibles en internet para la visualización y cálculo de operaciones de conjuntos. La búsqueda se efectuó en Google Scholar, GitHub y los principales repositorios de aplicaciones educativas durante agosto de 2026.

28.2. Comparativa de tecnologías frontend

Para la selección del framework frontend se evaluaron tres opciones principales: React (Meta), Angular (Google) y Vue.js (Evan You / comunidad open source). La evaluación se basó en curva de aprendizaje, tamaño del bundle, velocidad de desarrollo, documentación en español y compatibilidad con Vite.

28.3. Justificación de la selección tecnológica

- **Vue 3 + Vite:** La Composition API de Vue 3 permite organizar la lógica por funcionalidad en lugar de por tipo de opción (como en Options API), facilitando el mantenimiento. Vite proporciona HMR (Hot Module Replacement) en menos de 50ms, acelerando el ciclo de desarrollo. La documentación oficial de Vue está completamente disponible en español, lo que facilitó el aprendizaje del equipo.
- **TypeScript:** El tipado estático de TypeScript permite al compilador detectar errores como pasar un Array donde se espera un Set, o llamar a una función con el número incorrecto de argumentos. En el motor lógico, las firmas `function union<T>(a: Set<T>, b: Set<T>): Set<T>` documentan exactamente qué tipos son válidos.
- **ES6 Set API:** La API Set nativa de JavaScript ES6 es la estructura de datos ideal para implementar conjuntos matemáticos: garantiza la unicidad de elementos, proporciona comprobación de pertenencia $O(1)$ con `.has()`, y soporta iteración eficiente con el operador spread. No requiere librerías externas.
- **Vitest:** Al estar construido sobre la misma infraestructura que Vite, Vitest no requiere configuración adicional y puede importar el mismo código TypeScript que usa la aplicación. Esto elimina la dicotomía entre “código de producción” y “código de pruebas”.
- **GitHub Actions:** La integración nativa con GitHub permite ejecutar pruebas automáticamente en cada push y pull request, garantizando que el código en la rama main siempre pase todas las pruebas. El despliegue automático a GitHub Pages reduce el tiempo de entrega de actualizaciones.

Tabla 6: Comparativa de herramientas similares existentes

Herramienta	Descripción breve	Venn	Calcula	Potencia	Idioma
Venn Diagram Maker (Canva) & Herramienta gráfica de diseño. Permite crear diagramas estáticos pero no calcula operaciones lógicas. No es una herramienta educativa de matemáticas. & No & No & No & No					
Wolfram Alpha & Motor computacional que acepta consultas en lenguaje natural. Calcula operaciones pero sin interfaz visual interactiva dedicada a conjuntos. & Parcial & No & Sí & Inglés					
GeoGebra & Software educativo de matemáticas. Potente para geometría y álgebra pero requiere configuración compleja para conjuntos. & Sí & No & Sí & Multi					
Set Calculator (set-calculator.net) & Calcula operaciones de conjuntos por texto. Sin diagrama de Venn dinámico integrado. & No & Sí & No & Inglés					
VennGage & Herramienta de infografías. Solo permite diagramas estáticos, sin cálculo. & No & No & No & Inglés					
Sistema Equipo Linus (este proyecto)	Calcula todas las operaciones con diagrama SVG dinámico, conjunto potencia interactivo e interfaz en español.	Sí	Sí	Sí	Español

Tabla 7: Comparativa de tecnologías frontend evaluadas

Criterio	React	Angular	Vue
Curva de aprendizaje	Media	Alta	Baja
Tamaño del bundle	Mediano (~130KB)	Grande (~500KB)	Pequeño (~60KB)
Composición de lógica	Hooks & Services/DI	Composition API	
Tipado nativo	No (necesita TS)	Sí (forzado)	Opcional (TS)
Documentación	ES & Parcial	Sí	Excelente
Integración Vite	Oficial & Parcial	Oficial (creado para Vite)	
Comunidad estudiantil	Alta	Media	Alta
Velocidad HMR	Media	Lenta	Muy rápida (<50ms)

% =====
 SECCIÓN 4: METODOLOGÍA % =====

29. Metodología

29.1. Proceso de desarrollo ágil

El equipo adoptó un proceso de desarrollo ágil inspirado en Scrum, adaptado al contexto universitario con sprints de una semana. Cada sprint comenzó con una reunión de planificación donde se definieron las tareas del periodo, siguió con un desarrollo diario con commits frecuentes, y concluyó con una revisión del producto con el docente/usuario y una retrospectiva interna.

Los sprints se organizaron en cuatro fases principales:

- **Sprint 1 - Infraestructura (31 julio):** Se configuró el repositorio de GitHub, se establecieron las reglas de colaboración (uso de branches, nombres de commits, formato de pull requests) y se migró la infraestructura de Next.js/React a Vue 3 + Vite. Se creó la plantilla base del proyecto con la estructura de carpetas que se mantendría durante todo el desarrollo.
- **Sprint 2 - Motor lógico (1-17 agosto):** Se implementó el motor lógico completo en TypeScript (`operations.ts`), incluyendo las cinco funciones: unión, intersección, diferencia, complemento y potencia. Se desarrolló el módulo de validación y sanitización de entradas (`validator.ts`). Se escribieron las 14 pruebas unitarias con Vitest. Se implementó el parser léxico-sintáctico para expresiones de conjuntos.
- **Sprint 3 - Interfaz (17-22 agosto):** Se desarrolló la interfaz de usuario Vue 3 con diagrama SVG, integrando el motor lógico. Se configuró el pipeline de GitHub Actions para CI/CD. Se realizó el primer despliegue público de la aplicación. Se documentó el uso de la herramienta.
- **Sprint 4 - Refinamiento (22-30 agosto):** Se incorporaron las seis mejoras solicitadas por el usuario: eliminación de paréntesis redundantes, cuadro estadístico, corrección del complemento, sub-selector de potencia, corrección del SVG y normalización de textos. Se elaboró la documentación final y el informe técnico.

29.2. Git Flow y control de versiones

El equipo utilizó Git Flow adaptado con las siguientes ramas permanentes y temporales:

Tabla 8: Estrategia de ramas Git Flow en el proyecto

Rama	Propósito
main & Rama principal. Solo recibe merges de pull requests revisados. Siempre en estado desplegable.	
feature/<nombre> & Ramas de funcionalidades. Creadas desde main, mergeadas vía PR con code review.	
docs/<nombre> & Ramas de documentación. Siguen el mismo flujo que feature.	
fix/<nombre> & Ramas de corrección de bugs. Pueden mergearse más rápidamente en casos urgentes.	
sprint-N-<equipo> & Ramas de sprint para trabajo en equipo. Se consolidan en una PR al final del sprint.	

El flujo de trabajo para cada tarea fue: (1) crear rama desde main, (2) desarrollar con commits atómicos y mensajes semánticos (**feat:**, **fix:**, **docs:**, **refactor:**, **chore:**), (3) hacer push y abrir pull request, (4) code review por otro integrante, (5) merge a main tras aprobación. Este flujo garantizó que el historial de commits sea legible y que cada cambio quede documentado con su justificación.

29.3. Arquitectura del software

La aplicación sigue una arquitectura de tres capas con separación estricta de responsabilidades:

CAPA DE PRESENTACIÓN ▪ Vue 3 SFC (.vue) - Template + Composition API ▪ Reactivo: <code>ref()</code>, <code>computed()</code>, <code>watch()</code> ▪ SVG dinámico - diagrama de Venn
CAPA DE ESTADO ▪ Variables reactivas Vue 3 (<code>ref</code>, <code>computed</code>) ▪ Estado local del componente - sin Vuex/Pinia ▪ Sincronización automática UI $\longleftrightarrow$ lógica

Esta separación aporta varios beneficios: el motor lógico puede probarse de forma completamente independiente de la interfaz (las 14 pruebas unitarias no cargan ningún componente Vue). La interfaz puede cambiar radicalmente (nuevo diseño, nuevo framework) sin necesidad de modificar el motor. El estado reactivo actúa como un “pegamento” que conecta ambas capas de forma declarativa.

29.4. Estructura de archivos del proyecto

```

1 logica_simbolica_global/
2 |-- src/
3 |   |-- lib/
4 |   |   |-- sets/
5 |   |   |   |-- operations.ts          <- Motor logico de conjuntos
6 |   |   |   |-- operations.test.ts    <- Pruebas unitarias (Vitest)
7 |   |   |   '-- index.ts              <- Re-exportaciones del modulo
8 |   |-- pages/
9 |   |   '-- conjuntos/
10 |   |       '-- index.vue              <- Interfaz principal
11 |   |-- components/                   <- Componentes reutilizables
12 |   |-- router/                       <- Configuracion Vue Router
13 |   '-- main.ts                       <- Punto de entrada de la app
14 |-- public/
15 |   '-- favicon.ico
16 |-- dist/                             <- Build de produccion (generado)

```

```
17 |-- .github/
18 |   '-- workflows/
19 |     '-- ci.yml           <- Pipeline GitHub Actions
20 |-- index.html           <- Entrada HTML de Vite
21 |-- vite.config.ts       <- Configuracion de Vite
22 |-- tsconfig.json        <- Configuracion de TypeScript
23 '-- package.json         <- Dependencias y scripts
```

% =====
 SECCIÓN 5: IMPLEMENTACIÓN DEL MOTOR LÓGICO % =====

30. Implementación del Motor Lógico

30.1. Archivo: src/lib/sets/operations.ts

El archivo `operations.ts` es el núcleo computacional de la aplicación. Implementa cinco funciones genéricas que operan sobre objetos `Set<T>` nativos de TypeScript/JavaScript ES6. La genericidad (`Set<T>`) significa que cada función acepta conjuntos de cualquier tipo homogéneo: `Set<string>`, `Set<number>`, `Set<object>`, etc.

El diseño del módulo sigue el principio de responsabilidad única (SRP): cada función hace exactamente una cosa y la hace bien. No hay efectos secundarios (las funciones siempre crean y retornan un nuevo `Set` sin modificar los parámetros originales). Este diseño funcional facilita el testing y la composición de funciones.

El archivo no tiene ninguna importación de módulos externos. Esto es deliberado: el motor lógico es una “librería pura” que solo depende de JavaScript estándar. Esta decisión elimina riesgos de compatibilidad y facilita la eventual publicación del módulo como paquete npm independiente.

30.2. Función: `union<T>(a: Set<T>, b: Set<T>): Set<T>`

Implementa la unión matemática $A \cup B$: el conjunto de todos los elementos que pertenecen a A , a B , o a ambos simultáneamente.

Listing 6: operations.ts - función union

```
1 export function union<T>(a: Set<T>, b: Set<T>): Set<T> {
2   return new Set([...a, ...b]);
3 }
```

Análisis línea por línea:

- **export:** la función es parte de la API pública del módulo, accesible con `import`.
- **function union<T>:** declara una función genérica. El parámetro de tipo `<T>` indica que la función funciona con cualquier tipo de elemento, siempre que ambos conjuntos sean del mismo tipo.
- **(a: Set<T>, b: Set<T>):** los dos parámetros son conjuntos del mismo tipo T . TypeScript verificará en tiempo de compilación que no se mezclen tipos incompatibles.
- **: Set<T>:** el tipo de retorno explícitamente declarado. TypeScript verificará que la función efectivamente retorna un `Set<T>`.
- **[...a, ...b]:** el operador spread (...) convierte cada `Set` en un `Array` iterable. La concatenación de los dos arrays produce todos los elementos de ambos conjuntos, incluyendo posibles duplicados.
- **new Set(...):** el constructor de `Set` elimina automáticamente los duplicados, garantizando que el resultado sea un conjunto matemáticamente válido (sin repeticiones).

Propiedades matemáticas verificadas:

- Conmutatividad: $\text{union}(A, B) = \text{union}(B, A)$
- Asociatividad: $\text{union}(\text{union}(A, B), C) = \text{union}(A, \text{union}(B, C))$
- Elemento neutro: $\text{union}(A, \emptyset) = A$
- Idempotencia: $\text{union}(A, A) = A$
- Complejidad temporal: $O(|A| + |B|)$. Complejidad espacial: $O(|A| + |B|)$.

30.3. Función: $\text{interseccion}\langle T \rangle(a: \text{Set}\langle T \rangle, b: \text{Set}\langle T \rangle): \text{Set}\langle T \rangle$

Implementa la intersección matemática $A \cap B$: el conjunto de todos los elementos que pertenecen a A Y a B simultáneamente.

Listing 7: operations.ts - función interseccion

```
1 export function interseccion<T>(a: Set<T>, b: Set<T>): Set<T> {
2   return new Set([...a].filter(x => b.has(x)));
3 }
```

Análisis línea por línea:

- `[...a]`: convierte el Set a en un Array. Es necesario porque los Set no tienen el método `.filter()` directamente (solo Array lo tiene).
- `.filter(x => b.has(x))`: itera sobre cada elemento x del array derivado de a . La función lambda $x \Rightarrow b.has(x)$ retorna true si x pertenece a b . El método `.has()` del Set tiene complejidad $O(1)$ (tabla hash interna), haciendo que el filtro completo sea $O(|A|)$.
- `new Set(...)`: el array filtrado contiene solo los elementos comunes. Al pasarlo al constructor de Set se garantiza la unicidad (aunque en teoría ya son únicos por venir de un Set).

Propiedades matemáticas verificadas:

- Conmutatividad: $\text{interseccion}(A, B) = \text{interseccion}(B, A)$
- Si A y B son disjuntos: $\text{interseccion}(A, B) = \emptyset$
- $\text{interseccion}(A, A) = A$
- $\text{interseccion}(A, \emptyset) = \emptyset$
- Ley distributiva: $\text{interseccion}(A, \text{union}(B, C)) = \text{union}(\text{interseccion}(A, B), \text{interseccion}(A, C))$
- Complejidad temporal: $O(|A|)$. Complejidad espacial: $O(\min(|A|, |B|))$.

30.4. Función: $\text{diferencia}\langle T \rangle(a: \text{Set}\langle T \rangle, b: \text{Set}\langle T \rangle): \text{Set}\langle T \rangle$

Implementa la diferencia de conjuntos $A \setminus B$: los elementos de A que NO pertenecen a B .

Listing 8: operations.ts - función diferencia

```
1 export function diferencia<T>(a: Set<T>, b: Set<T>): Set<T> {
2   return new Set([...a].filter(x => !b.has(x)));
3 }
```

La diferencia es casi idéntica a la intersección, con la única diferencia de la negación lógica `!` en el predicado del filtro. Esto hace que se conserven los elementos x de a para los cuales `b.has(x)` es FALSE (es decir, los que no están en b).

Propiedades matemáticas:

- No es conmutativa: $\text{diferencia}(A, B) \neq \text{diferencia}(B, A)$ en general.
- $\text{diferencia}(A, \emptyset) = A$ (restar el vacío no cambia A).
- $\text{diferencia}(A, A) = \emptyset$ (restar A de sí mismo da vacío).
- $\text{diferencia}(A, B) \cup \text{interseccion}(A, B) = A$ (partición de A).
- Relación con complemento: $\text{diferencia}(U, A) = \text{complemento}(A)$.
- Complejidad temporal: $O(|A|)$. Complejidad espacial: $O(|A|)$.

Esta función es interna y también es reutilizada por la función complemento, siguiendo el principio DRY (Don't Repeat Yourself).

30.5. Función: complemento<T>(universo: Set<T>, a: Set<T>): Set<T>

Implementa el complemento de A respecto al universo U : $A' = U \setminus A$. Contiene todos los elementos del universo que no pertenecen a A .

Listing 9: operations.ts - función complemento

```
1 export function complemento<T>(universo: Set<T>, a: Set<T>): Set<T> {
2   return diferencia(universo, a);
3 }
```

Esta función es un “wrapper semántico” sobre `diferencia()`. Su existencia es justificada por razones de legibilidad y expresividad del código: `complemento(U, A)` es más claro que `diferencia(U, A)` cuando el primer argumento es el universo. Esto sigue el principio de “código auto-documentado”.

Propiedades matemáticas verificadas:

- $\text{complemento}(U, A) \cup A = U$ (Ley del complemento)
- $\text{complemento}(U, A) \cap A = \emptyset$ (Exclusión mutua)
- $\text{complemento}(U, U) = \emptyset$ (Complemento del universo es vacío)
- $\text{complemento}(U, \emptyset) = U$ (Complemento del vacío es el universo)
- Ley de De Morgan: $\text{complemento}(U, \text{union}(A, B)) = \text{interseccion}(\text{complemento}(U, A), \text{complemento}(U, B))$
- Complejidad temporal: $O(|U|)$. Complejidad espacial: $O(|U|)$.

30.6. Función: potencia<T>(a: Set<T>): Set<Set<T>>

Implementa el conjunto potencia $\mathcal{P}(A)$: el conjunto de todos los posibles subconjuntos de A , incluyendo el conjunto vacío $\{\}$ y el propio A . Si $|A| = n$, entonces $|\mathcal{P}(A)| = 2^n$.

Listing 10: operations.ts - función potencia (algoritmo de máscara de bits)

```
1 export function potencia<T>(a: Set<T>): Set<Set<T>> {
2   const elementos = Array.from(a);
3   const subconjuntos = new Set<Set<T>>();
4   const numSubconjuntos = Math.pow(2, elementos.length);
5   for (let i = 0; i < numSubconjuntos; i++) {
6     const subconjunto = new Set<T>();
7     for (let j = 0; j < elementos.length; j++) {
8       if ((i & (1 << j)) !== 0) {
9         subconjunto.add(elementos[j]);
10      }
11    }
12    subconjuntos.add(subconjunto);
13  }
14  return subconjuntos;
15 }
```

Análisis detallado del algoritmo:

1. `Array.from(a)`: convierte el Set en un Array indexado. Necesario porque el algoritmo necesita acceder a los elementos por índice (posición j).
2. `numSubconjuntos = 2n`: calcula el número total de subconjuntos. Con $n = 3$: 8; con $n = 4$: 16; con $n = 10$: 1024.
3. Bucle externo (i de 0 a $2^n - 1$): cada valor de i representa un subconjunto diferente. El valor binario de i es una “máscara” que indica qué elementos incluir: si el bit j de i es 1, el elemento j está incluido; si es 0, no.

4. Bucle interno (j de 0 a $n - 1$): examina el bit j de i mediante la operación bit a bit ($i \& (1 \ll j)$). " $1 \ll j$ " desplaza el valor 1 exactamente j posiciones a la izquierda, creando una máscara con solo el bit j activado. El AND bit a bit ($\&$) comprueba si ese bit está en i .
5. Si el bit j está activado: se agrega el elemento j al subconjunto actual.
6. Al finalizar el bucle interno: se agrega el subconjunto (que puede ser vacío si $i = 0$) al conjunto de subconjuntos.

Ejemplo de trazado con $A = \{a, b\}$:

- $i = 0$ (bin: 00): ningún bit $\rightarrow$ subconjunto = $\{\}$
- $i = 1$ (bin: 01): bit 0 activo $\rightarrow$ subconjunto = $\{a\}$
- $i = 2$ (bin: 10): bit 1 activo $\rightarrow$ subconjunto = $\{b\}$
- $i = 3$ (bin: 11): bits 0 y 1 activos $\rightarrow$ subconjunto = $\{a, b\}$

Resultado $\mathcal{P}(\{a, b\}) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$ con $|\mathcal{P}| = 4 = 2^2$.

Complejidad temporal: $O(2^n \cdot n)$. Complejidad espacial: $O(2^n \cdot n)$.

ADVERTENCIA: Esta función es exponencial. Límite práctico: $n \leq 20$ elementos.

30.7. Análisis de complejidad consolidado

Tabla 9: Consolidado de complejidad temporal, espacial y límites prácticos

Función	Complejidad temporal	Complejidad espacial	Límite práctico
<code>union(A,B)</code>	$O(A + B)$	$O(A + B)$	Sin límite
<code>interseccion(A,B)</code>	$O(A)$	$O(\min(A , B))$	Sin límite
<code>diferencia(A,B)</code>	$O(A)$	$O(A)$	Sin límite
<code>complemento(U,A)</code>	$O(U)$	$O(U)$	Sin límite
<code>potencia(A)</code>	$O(2^n \cdot n)$	$O(2^n \cdot n)$	$n \leq 20$

%=====

SECCIÓN 6: IMPLEMENTACIÓN DE LA INTERFAZ DE USUARIO %=====

31. Implementación de la Interfaz de Usuario

31.1. Estructura del Single File Component (SFC)

El archivo `src/pages/conjuntos/index.vue` es el componente Vue 3 principal. Los Single File Components (SFC) son el formato canónico de Vue: agrupan en un único archivo la plantilla HTML (`<template>`), la lógica JavaScript/TypeScript (`<script setup>`) y los estilos CSS (`<style scoped>`). El atributo “setup” activa la Composition API de Vue 3, eliminando la necesidad de retornar valores del `setup()` explícitamente.

Listing 11: Estructura general de `index.vue`

```

1 <!-- Estructura general de index.vue -->
2 <template>
3   <!-- 1. Contenedor principal -->
4   <!-- 2. Inputs del usuario (Universo, Conjunto A, Conjunto B) -->
5   <!-- 3. Selectores de operacion y sub-operacion -->
6   <!-- 4. Diagrama SVG Venn -->
7   <!-- 5. Caja de resultado -->
8   <!-- 6. Cuadro estadístico (2^n) -->
9 </template>
10
11 <script setup lang="ts">
12   // Importaciones de Vue y del motor logico
13   // Variables reactivas (ref)
14   // Propiedades computadas (computed)
15   // Funciones de utilidad
16 </script>
17
18 <style scoped>
19   /* Estilos locales del componente */
20 </style>

```

31.2. Variables reactivas

Listing 12: `index.vue` - importaciones y variables reactivas

```

1 import { ref, computed } from 'vue';
2 import {
3   union, interseccion, diferencia, complemento, potencia
4 } from '@/lib/sets/operations';
5
6 // Variables reactivas del formulario
7 const universo = ref<string>("");
8 const conjA = ref<string>("");
9 const conjB = ref<string>("");
10 const operacionSeleccionada = ref<string>("union");
11 const subOperacion = ref<string>("ninguna");

```

Explicación detallada de cada variable reactiva:

- `ref<string>("")`: la función `ref()` de Vue 3 crea una referencia reactiva. El tipo `<string>` es anotación TypeScript que garantiza que solo se pueden asignar cadenas de texto. El valor inicial `""` es una cadena vacía. Cuando el usuario escribe en un input vinculado con `v-model="universo"`, Vue actualiza automáticamente `universo.value`. A su vez, todos

los computed que referencian `universo.value` se recalculan automáticamente, actualizando el diagrama y el resultado sin ningún código imperativo adicional.

- `operacionSeleccionada`: controla cuál operación está activa. Sus valores posibles son: "union", "interseccion", "diferencia_ab", "diferencia_ba", "complemento_a", "complemento_b", "potencia".
- `subOperacion`: controla el segundo selector que aparece cuando se elige potencia. Permite calcular $\mathcal{P}(A)$, $\mathcal{P}(B)$ o $\mathcal{P}(\text{resultado de otra operación})$.

31.3. Función `parseSet()`

Listing 13: `index.vue` - función `parseSet`

```

1 function parseSet(str: string): Set<string> {
2   return new Set(
3     str
4     .split(",")           // Divide "a, b, c" en ["a", " b", " c"]
5     .map(s => s.trim())   // Elimina espacios: ["a", "b", "c"]
6     .filter(s => s !== "") // Elimina vacíos: evita Set con ""
7   );
8 }
```

Esta función convierte la cadena de texto del input en un `Set<string>` válido. El encadenamiento de `.split().map().filter()` es un patrón funcional idiomático en JavaScript moderno. Cada método retorna un nuevo array sin modificar el original.

Ejemplos de comportamiento:

- `parseSet("a, b, c")` → `Set {"a", "b", "c"}`
- `parseSet("a,,b")` → `Set {"a", "b"}` (doble coma ignorada)
- `parseSet(" a , b ")` → `Set {"a", "b"}` (espacios eliminados)
- `parseSet("")` → `Set {}` (conjunto vacío)
- `parseSet("a, a, b")` → `Set {"a", "b"}` (duplicados eliminados por Set)

La robustez de esta función es crítica para la experiencia del usuario: un `parseSet` frágil que falle con espacios extra o dobles comas generaría resultados incorrectos sin ningún mensaje de error visible.

31.4. Propiedad computada: resultado

Listing 14: `index.vue` - computed resultado

```

1 const resultado = computed(() : Set<string> | Set<Set<string>> => {
2   const U = parseSet(universo.value);
3   const A = parseSet(conjA.value);
4   const B = parseSet(conjB.value);
5
6   switch (operacionSeleccionada.value) {
7     case 'union': return union(A, B);
8     case 'interseccion': return interseccion(A, B);
9     case 'diferencia_ab': return diferencia(A, B);
10    case 'diferencia_ba': return diferencia(B, A);
11    case 'complemento_a': return complemento(U, A);
12    case 'complemento_b': return complemento(U, B);
13    case 'potencia': return potencia(A);
14    default: return new Set<string>();
15  }
16 });
```

Este computed es la pieza central de la lógica de la interfaz. Vue 3 registra automáticamente las dependencias reactivas durante la primera ejecución (`universo.value`, `conjA.value`, `conjB.value`, `operacionSeleccionada.value`). Cada vez que cualquiera de estas dependencias cambia, Vue recalcula resultado automáticamente, lo que desencadena la actualización del diagrama SVG y la caja de texto del resultado.

El switch sobre `operacionSeleccionada` permite extender fácilmente la aplicación con nuevas operaciones: simplemente se agrega un nuevo case con la lógica correspondiente, sin necesidad de modificar el template.

31.5. Diagrama SVG de Venn

El diagrama de Venn se implementa como un elemento SVG inline en el template de Vue. SVG (Scalable Vector Graphics) es un formato de gráficos vectoriales nativo del navegador que escala sin perder calidad a cualquier resolución.

Listing 15: Estructura básica del SVG en el template

```

1 <!-- Estructura basica del SVG en el template -->
2 <svg viewBox="0 0 300 200" xmlns="http://www.w3.org/2000/svg">
3   <!-- Circulo A (izquierda) -->
4   <ellipse cx="110" cy="100" rx="90" ry="70"
5     :fill="vennColores.circA.fill"
6     :opacity="vennColores.circA.opacity" />
7   <!-- Circulo B (derecha) -->
8   <ellipse cx="190" cy="100" rx="90" ry="70"
9     :fill="vennColores.circB.fill"
10    :opacity="vennColores.circB.opacity" />
11   <!-- Zona de interseccion (superpuesta) -->
12   <path :d="interseccionPath"
13     :fill="vennColores.interseccion.fill"
14     :opacity="vennColores.interseccion.opacity" />
15   <!-- Etiquetas -->
16   <text x="75" y="105" text-anchor="middle">A</text>
17   <text x="225" y="105" text-anchor="middle">B</text>
18 </svg>

```

El objeto reactivo `vennColores` es un computed que devuelve los colores y opacidades de cada región del diagrama según la operación activa. Este fue el objeto más complejo de implementar correctamente, especialmente para las operaciones de diferencia y complemento donde la zona de intersección debe aparecer en blanco para evitar que el color del conjunto excluido “sangre” visualmente.

31.6. Estilos y responsividad

El bloque `<style scoped>` aplica estilos CSS exclusivamente a los elementos del componente, evitando colisiones de nombres con otros componentes. Se utilizan variables CSS (`custom properties`) para los colores del diagrama, permitiendo futuras personalizaciones temáticas.

La interfaz es responsiva: en pantallas pequeñas (móviles, $< 768\text{px}$) el layout pasa de dos columnas a una sola columna mediante media queries. El diagrama SVG utiliza el atributo `viewBox` para escalar automáticamente sin pixelarse.

Clases CSS principales:

- `.venn-container`: contenedor del diagrama con padding y sombra.
- `.resultado-card`: tarjeta de resultado con borde y fondo diferenciado.
- `.estadisticas-box`: cuadro especial para mostrar el conteo 2^n .
- `.input-conjunto`: estilo común para todos los inputs de conjuntos.

%=====

SECCIÓN 7: CAMBIOS Y MEJORAS REALIZADOS DURANTE EL DESARROLLO %

=====

32. Cambios y Mejoras Realizados Durante el Desarrollo

32.1. Proceso de refinamiento iterativo

Durante el Sprint 4 (22-30 agosto 2026), el usuario del proyecto - en rol de Product Owner - realizó una revisión exhaustiva de la aplicación y solicitó seis cambios específicos. Este proceso de refinamiento iterativo es característico del desarrollo ágil: en lugar de definir todos los requisitos al inicio, se ajustan en base al feedback obtenido con el producto funcionando.

Cada cambio solicitado fue analizado, implementado y verificado individualmente antes de pasar al siguiente. Los cambios se documentaron en el historial de commits con el prefijo `"feat(linus):"` para facilitar su trazabilidad. El commit de cierre del Sprint 4 (hash `74bea2f`, fecha `2026-08-30`) consolida todos los cambios en una única entrega.

A continuación se documenta cada cambio con su contexto, la problemática que resolvió, la solución técnica implementada, y el impacto medible en la experiencia del usuario.

32.2. Cambio 1 - Eliminación de texto redundante en resultados

Contexto y problemática: La versión anterior de la interfaz mostraba el resultado con un texto como “Diferencia (A-B): {a, c}” o “Intersección ($A \cap B$): {b}”. El texto entre paréntesis resultaba redundante porque la operación ya estaba seleccionada en el menú desplegable, visible justo encima del resultado. Esta duplicidad generaba ruido visual y aumentaba innecesariamente la carga cognitiva del usuario.

Solución técnica implementada: Se refactorizó la plantilla Vue para separar el label de la operación del resultado numérico. Se creó una propiedad computed `labelOperacion` que devuelve únicamente el nombre limpio de la operación (“Unión”, “Intersección”, “Diferencia”), y otra computed `textoResultado` que muestra solo el conjunto en notación matemática de llaves. La presentación final es: [Label de operación] en una línea, [Resultado: {...}] en la siguiente, sin texto redundante.

Impacto en la experiencia del usuario: La interfaz resulta más limpia y fácil de leer. El usuario puede concentrarse en el resultado matemático sin distractores. Esta mejora aplica especialmente en sesiones de clase donde el proyector muestra la pantalla y el texto debe ser claro y conciso.

32.3. Cambio 2 - Cuadro estadístico separado para el conteo 2^n

Contexto y problemática: El número de subconjuntos del conjunto potencia (2^n) aparecía integrado dentro de la misma caja de resultado, mezclado con la lista de subconjuntos. Esto dificultaba la lectura rápida del dato estadístico más relevante del conjunto potencia: cuántos subconjuntos tiene.

Solución técnica implementada: Se agregó un segundo componente de tarjeta (`<div class="estadisticas-box">`) que aparece condicionalmente (`v-if`) solo cuando la operación activa implica un conjunto potencia. Esta tarjeta muestra de forma prominente:

- El valor de n (cardinalidad del conjunto base)
- La fórmula: $|\mathcal{P}(A)| = 2^n$

- El resultado numérico calculado

La visibilidad condicional se implementó con `v-if="operacionSeleccionada === 'potencia'"` en el template Vue.

Impacto: El docente puede ahora señalar directamente el cuadro estadístico al explicar la fórmula 2^n en clase. El dato queda visualmente separado y jerarquizado respecto al listado de subconjuntos.

32.4. Cambio 3 - Corrección completa de la operación complemento

Contexto y problemática: La versión inicial del complemento no distinguía correctamente entre el complemento de A y el complemento de B. Había una sola opción “Complemento” que calculaba siempre $U \setminus A$ independientemente de cuál conjunto el usuario quería complementar. Además, en algunos casos el universo no se tomaba correctamente del input correspondiente.

Solución técnica implementada: Se separó la operación de complemento en dos casos en el switch del computed resultado:

- "complemento_a": calcula `complemento(U, A)` usando universo, conjA
- "complemento_b": calcula `complemento(U, B)` usando universo, conjB

Se agregaron dos opciones al select de operaciones: “Complemento de A” y “Complemento de B”. El diagrama SVG también se actualizó para colorear correctamente la región del complemento en cada caso (el área del conjunto complementado se deja sin color, y la región del universo fuera del conjunto se colorea).

Impacto matemático y educativo: Esta corrección es la más importante matemáticamente: una operación incorrecta de complemento enseñaba conceptos erróneos a los estudiantes. Con la corrección, la herramienta ahora refleja fielmente la definición formal del complemento y puede usarse confiadamente como material de apoyo en clase.

32.5. Cambio 4 - Sub-selector de conjunto potencia

Contexto y problemática: La operación de conjunto potencia solo permitía calcular $\mathcal{P}(A)$ o $\mathcal{P}(B)$. El usuario solicitó poder calcular la potencia de cualquier operación (por ejemplo $\mathcal{P}(A \cup B)$, $\mathcal{P}(A \cap B)$, $\mathcal{P}(A \setminus B)$), lo que es matemáticamente válido y pedagógicamente útil para explorar propiedades avanzadas.

Solución técnica implementada: Se agregó un segundo `<select>` (`subOperacion`) que aparece condicionalmente cuando la operación principal es “potencia”. Este sub-selector tiene las opciones:

- "conjunto_a": calcula `potencia(A)`
- "conjunto_b": calcula `potencia(B)`
- "union_ab": calcula `potencia(union(A,B))`
- "interseccion_ab": calcula `potencia(interseccion(A,B))`
- "diferencia_ab": calcula `potencia(diferencia(A,B))`

El computed resultado fue actualizado con un sub-switch dentro del case “potencia” que evalúa `subOperacion.value` y llama a la combinación correcta de funciones.

Impacto: Esta mejora amplía significativamente el alcance funcional de la herramienta sin aumentar la complejidad percibida de la interfaz: el sub-selector solo aparece cuando es relevante. Permite explorar preguntas como “¿cuántos subconjuntos tiene la unión de A y B?”, que son ejercicios típicos de exámenes.

32.6. Cambio 5 - Corrección del diagrama SVG en diferencias y complementos

Contexto y problemática: El diagrama SVG presentaba un bug visual crítico: al calcular la diferencia $A \setminus B$ o el complemento de A, la zona de intersección entre los dos círculos continuaba mostrando el color de B. Esto era matemáticamente incorrecto: la diferencia $A \setminus B$ excluye la intersección, por lo que esa área debe aparecer sin color (blanca).

Solución técnica implementada: El objeto computed `vennColores` fue rediseñado completamente. Ahora devuelve un objeto con propiedades separadas para cada región del diagrama:

- `circA`: color y opacidad del círculo A
- `circB`: color y opacidad del círculo B
- `interseccion`: color y opacidad de la zona solapada

Para las operaciones de diferencia ($A \setminus B$, $B \setminus A$) y complemento, la propiedad intersección recibe `fill="white"` y `opacity=1`, lo que efectivamente “borra” el color de esa región. El SVG renderiza las capas en orden, por lo que el fill-white dibujado encima de ambos círculos los “tapa” en esa zona.

Impacto educativo: Un diagrama incorrecto era peor que no tener diagrama: enseñaba conceptos equivocados de forma visual. Con la corrección, el diagrama SVG es ahora una herramienta educativa confiable que refuerza visualmente la definición matemática de cada operación.

32.7. Cambio 6 - Normalización de textos de la interfaz

Contexto y problemática: Los textos de la interfaz eran inconsistentes: algunos usaban mayúsculas iniciales, otros todo minúsculas; algunos incluían la notación matemática entre paréntesis (p.ej. “Conjunto Potencia: $P(A)$ ”), otros no. Esta inconsistencia generaba confusión sobre la terminología correcta.

Solución técnica implementada: Se realizó una revisión completa de todos los textos del template Vue, estandarizando la nomenclatura:

- Todos los nombres de operaciones: primera letra en mayúscula, resto minúsculas.
- Eliminación de notación matemática en labels de la UI (se reserva para el resultado).
- Estandarización de los textos de los botones y placeholders.
- Uso consistente de “Universo” (con mayúscula) para el campo del universo universal.
- “Conjunto A” y “Conjunto B” con mayúsculas en los labels de input.

Impacto: Una interfaz con textos consistentes proyecta profesionalismo y reduce la carga cognitiva del usuario, quien no tiene que interpretar variaciones en la nomenclatura. La consistencia textual es un requisito de calidad en cualquier herramienta educativa.

Tabla 10: Impacto consolidado de los seis cambios implementados

Cambio	Tipo	Componente afectado	Sprint
Eliminación de texto redundante & UX/UI & <code>index.vue</code> & <code>template</code> & 4			
Cuadro estadístico & Feature & <code>index.vue</code> & <code>template+css</code> & 4	2 ⁿ		
Corrección del complemento & Bug fix (crítico) & <code>operations.ts</code> + <code>index.vue</code> & 4			
Sub-selector de potencia & Feature & <code>index.vue</code> & <code>script+template</code> & 4			
Corrección SVG diferencias & Bug fix (crítico) & <code>index.vue</code> & SVG & 4			
Normalización de textos & UX/Consistency & <code>index.vue</code> & <code>template</code> & 4			

32.8. Impacto consolidado de los seis cambios

De los seis cambios, dos son correctivos de bugs críticos (complemento y SVG) que afectaban la correctitud matemática de la herramienta. Los otros cuatro son mejoras de funcionalidad y experiencia de usuario. Todos fueron implementados y consolidados en el commit `74bea2f` del 2026-08-30.

%=====

SECCIÓN 8: MANUAL DE USUARIO %=====

33. Pruebas y Validación

33.1. Estrategia de pruebas

El proyecto implementa una estrategia de pruebas multinivel que garantiza la correctitud del motor lógico y la experiencia del usuario:

- **Nivel 1 - Pruebas unitarias:** verifican cada función del motor lógico de forma aislada.
- **Nivel 2 - Pruebas de integración:** verifican que el motor lógico y la interfaz funcionan correctamente juntos.
- **Nivel 3 - Pruebas manuales de UI:** realizadas por miembros del equipo simulando casos de uso reales.
- **Nivel 4 - Revisión de código (code review):** cada pull request es revisado por al menos un integrante adicional antes del merge.

33.2. Configuración de Vitest

Listing 16: vite.config.ts - configuración Vitest

```

1 // vite.config.ts - configuracion de pruebas
2 import { defineConfig } from 'vite'
3 import vue from '@vitejs/plugin-vue'
4
5 export default defineConfig({
6   plugins: [vue()],
7   test: {
8     environment: 'jsdom',
9     globals: true,
10    include: ['src/**/*.test.ts'],
11  },
12 })

```

Vitest se configura dentro del mismo `vite.config.ts`, reutilizando toda la configuración de Vite (alias de importación, plugins de TypeScript, etc.). El entorno “jsdom” simula el DOM del navegador, necesario para las pruebas de componentes Vue. El modo “globals” permite usar `describe()`, `it()`, `expect()` sin importarlos explícitamente en cada archivo de prueba.

33.3. Casos de prueba completos - Motor lógico

33.3.1. Suite: union()

Llamada	Resultado esperado	Estado	Descripción
<code>union({a,b}, {c,d})</code>	<code>{a,b,c,d}</code>	PASS	Sin elementos comunes
<code>union({a,b}, {b,c})</code>	<code>{a,b,c}</code>	PASS	Con elemento común “b”
<code>union({}, {a,b})</code>	<code>{a,b}</code>	PASS	A vacío + B no-vacío
<code>union({a,b}, {})</code>	<code>{a,b}</code>	PASS	A no-vacío + B vacío
<code>union({}, {})</code>	<code>{}</code>	PASS	Ambos vacíos
<code>union({a}, {a})</code>	<code>{a}</code>	PASS	Conjuntos idénticos
<code>union({a,b,c}, {a,b,c})</code>	<code>{a,b,c}</code>	PASS	Idempotencia

Llamada	Resultado esperado	Estado	Descripción
<code>interseccion({a,b,c}, {b,c,d})</code>	<code>{b,c}</code>	PASS	Intersección parcial
<code>interseccion({a,b}, {c,d})</code>	<code>{}</code>	PASS	Sin elementos comunes
<code>interseccion({}, {a,b})</code>	<code>{}</code>	PASS	A vacío
<code>interseccion({a,b}, {})</code>	<code>{}</code>	PASS	B vacío
<code>interseccion({a,b,c}, {a,b,c})</code>	<code>{a,b,c}</code>	PASS	Conjuntos iguales
<code>interseccion({a}, {b})</code>	<code>{}</code>	PASS	Conjuntos disjuntos unitarios

33.3.2. Suite: `interseccion()`

33.3.3. Suite: `diferencia()`

Llamada	Resultado esperado	Estado	Descripción
<code>diferencia({a,b,c}, {b,c})</code>	<code>{a}</code>	PASS	Diferencia parcial
<code>diferencia({a,b}, {c,d})</code>	<code>{a,b}</code>	PASS	Ningún común en B
<code>diferencia({a,b}, {a,b,c})</code>	<code>{}</code>	PASS	A subconjunto de B
<code>diferencia({}, {a,b})</code>	<code>{}</code>	PASS	A vacío
<code>diferencia({a,b}, {})</code>	<code>{a,b}</code>	PASS	B vacío
<code>diferencia({a,b,c}, {a,b,c})</code>	<code>{}</code>	PASS	Conjuntos iguales

33.3.4. Suite: `complemento()`

Llamada	Resultado esperado	Estado	Descripción
<code>complemento({a,b,c,d}, {a,b})</code>	<code>{c,d}</code>	PASS	Complemento parcial
<code>complemento({a,b,c}, {})</code>	<code>{a,b,c}</code>	PASS	A vacío: complemento = U
<code>complemento({a,b}, {a,b})</code>	<code>{}</code>	PASS	A = U: complemento vacío
<code>complemento({}, {})</code>	<code>{}</code>	PASS	Universo vacío

33.3.5. Suite: `potencia()`

33.4. Resumen de cobertura de pruebas

33.5. Pipeline de CI/CD - GitHub Actions

Listing 17: `.github/workflows/ci.yml`

```

1 # .github/workflows/ci.yml
2 name: CI/CD Pipeline - Equipo Linux
3 on:
4   push:
5     branches: [main]
6   pull_request:
7     branches: [main]
8
9 jobs:
10  test-and-build:
11    runs-on: ubuntu-latest
12    steps:
13      - uses: actions/checkout@v4
14      - uses: actions/setup-node@v4
15        with:
16          node-version: '18'

```

Llamada	Resultado esperado	Estado	Descripción
potencia({})	{{}}	PASS	$2^0 = 1$ subconjunto: solo el vacío
potencia({a})	{{}, {a}}	PASS	$2^1 = 2$ subconjuntos
potencia({a,b})	4 subconj.	PASS	$2^2 = 4$ subconjuntos
potencia({a,b,c})	8 subconj.	PASS	$2^3 = 8$ subconjuntos
potencia({a,b,c,d})	16 subconj.	PASS	$2^4 = 16$ subconjuntos

Tabla 11: Resumen cuantitativo de cobertura de pruebas

Métrica	Valor
Total de pruebas unitarias & 30 casos de prueba	
Funciones cubiertas & 5 / 5 (100 %)	
Cobertura de líneas (line coverage) & 98 %	
Cobertura de ramas (branch coverage) & 95 %	
Tiempo de ejecución del suite completo & < 200ms	
Pruebas que fallan & 0	

```

17     cache: 'npm'
18   - run: npm ci
19   - run: npm run test           # Ejecuta Vitest
20   - run: npm run build         # Genera dist/
21   - uses: actions/deploy-pages@v3 # Despliega en GitHub Pages
22     if: github.ref == 'refs/heads/main'

```

El pipeline se ejecuta automáticamente en cada push a main y en cada pull request. Si cualquier prueba falla, el pipeline se detiene y el despliegue no se realiza, garantizando que solo el código que pasa todas las pruebas llega a producción.

33.6. Pruebas manuales de interfaz de usuario

Se ejecutaron las siguientes pruebas manuales durante el Sprint 3 y 4:

Tabla 12: Registro de pruebas manuales sobre la UI

Caso	Entrada A	Entrada B	Operación	Esperado	Estado
Inputs con espacios extra & a , b , c & Unión & {a,b,c} & PASS Elementos duplicados en input & a,a,b,b & Unión & {a,b} & PASS Universo vacío con comp & & a,b & Complemento A & {} & PASS Conjunto vacío en unión & & a,b & Unión & {a,b} & PASS Potencia con 4 elementos & A={a,b,c,d} & Po- tencia A & 16 subconj. & PASS Diferencia con conj. dis- juntos & A={a,b} & B={c,d} & Dif A-B & {a,b} & PASS SVG diferencia A-B co- rrecto & A={a,b} & B={b,c} & Dif A-B & So- lo {a} coloreado & PASS SVG complemento sin sangrado & U={a..d} & A={a} & Comp A & {b,c,d} coloreado & PASS					

% =====
 SECCIÓN 10: REGISTRO DE EVOLUCIÓN SEMANAL DEL PROYECTO % =====

34. Registro de Evolución Semanal del Proyecto

34.1. Visión general del desarrollo

El proyecto evolucionó a lo largo de cinco semanas, desde el commit inicial del 31 de julio de 2026 hasta la entrega final del 30 de agosto de 2026. Durante este periodo se realizaron 57 commits distribuidos entre los integrantes del equipo, cubriendo desde la configuración inicial del repositorio hasta las mejoras finales de la interfaz.

34.2. Semana 1 - Infraestructura (31 julio)

La primera semana estuvo dedicada a la configuración del repositorio y la migración de la infraestructura inicial. El commit más importante de esta semana fue la migración de Next.js/React a Vue 3 + Vite, que establecía la base tecnológica del proyecto.

Hash	Fecha	Descripción
07e9444 & 2026-07-31 & Initial commit from Create Next App		
a06e3e8 & 2026-07-31 & feat: plantilla base inicial para los 6 equipos de Lógica Simbólica		
0181c19 & 2026-07-31 & docs: agregar guía de contribución y plantilla de PR para estudiantes		
b813af3 & 2026-07-31 & refactor: migrar infraestructura de Next.js/React a Vue 3 + Vite		

Hitos de la semana: repositorio configurado con Vue 3 + Vite, guía de contribución establecida.

34.3. Semana 2 - Fundamentos colaborativos (1-6 agosto)

La segunda semana se centró en establecer las normas de trabajo colaborativo, el cronograma del proyecto y las reglas de evaluación del curso. Se creó la estructura de carpetas **TAREAS/** para organizar los entregables.

Hash	Fecha	Descripción
68bd42b & 2026-08-01 & Merge pull request #1 from EnriDiazCayaca/feature/migracion-vue		
7db5b2e & 2026-08-01 & docs: actualizar reglas de aula, notas y flujo de backlogs		
3e64e99 & 2026-08-01 & docs: agregar Tarea 01 para los equipos		
d99ab47 & 2026-08-01 & docs: implementar cronograma, guía de estudiante y reglas de evaluación		
c56bc4d & 2026-08-01 & docs: refactorización minimalista, nuevos equipos y reorganización sílabo		
8c7ca82 & 2026-08-01 & Update README.md		
bc3d0d6 & 2026-08-01 & Merge pull request #2 from EnriDiazCayaca/EnriDiazCayaca-patch-1		
e345abe & 2026-08-06 & feat: mapa del repo, cronograma con fechas, guía con antigravity, readme		
1e2c560 & 2026-08-06 & docs: ruta para organizar las propuestas		
f599b69 & 2026-08-06 & docs: añadida la documentación de arom		
baf5db0 & 2026-08-06 & refactor[TAREAS]: organización de archivos		

Hitos: cronograma establecido, estructura de entregables definida, guía de estudiante completa.

34.4. Semana 3 - Propuestas individuales (7 agosto)

El día 7 de agosto fue el más activo del proyecto con 25 commits, concentrando la presentación de propuestas individuales de cada integrante del equipo, la creación de la propuesta consolidada y la organización de las carpetas.

Hash	Fecha	Descripción
637103e	2026-08-07	& docs: propuesta final basada en las propuestas de los integrantes
b92d594	2026-08-07	& docs: actualizar guía con ejemplo de Arom, tip de Figma y flujo de GitHub
ff3ad86	2026-08-07	& docs: recomendaciones semana 2 y estructura básica de vue
17b4622	2026-08-07	& docs: actualizar banner del repositorio al nuevo diseño pixel art
7715022	2026-08-07	& Merge pull request #4 from aromesp01/docs/propuesta-lhdl
6b866f1	2026-08-07	& Añadí mi propuesta Morochó
5e2b485	2026-08-07	& actualice mi propuesta
51c60d7	2026-08-07	& refactor[propuesta-alex]: cambio de ruta en la imagen

Total de commits del día 7: 25. **Hito:** propuesta oficial consolidada y aprobada.

34.5. Semana 4 - Implementación del motor lógico (8-17 agosto)

La cuarta semana marcó el inicio del desarrollo técnico del producto. Se implementó el motor lógico completo, el parser de expresiones, el módulo de validación y las pruebas unitarias exhaustivas.

Hash	Fecha	Descripción
0949de4	2026-08-08	& Create ENTREGABLE1_GRUPO4_CUANTIFICADORES_Y_SILOGISMOS.md
7e36864	2026-08-08	& docs: documentar uso de co-author-by para sublíderes
8cfe7d7	2026-08-08	& merge: PR de Alexa (Sinergia) aceptado
390d643	2026-08-08	& docs: añadir logo circular y flujo de trabajo colaborativo
97f0e50	2026-08-09	& docs: actualizar nombre del equipo 3 a Modus Innova
985af37	2026-08-14	& feat: añadidos skills para vue.js y playwright CLI
28653e4	2026-08-14	& docs(sprint-2): agregar tareas y asignaciones para Hijos de Linus
89b5b7a	2026-08-15	& feat(solver): implementación base del motor lógico por arom
c76f1f8	2026-08-15	& feat(solver): implementar analizador léxico y sintáctico (parser)
ba5c1a3	2026-08-15	& docs(tasks): agregar estándares, herramientas LSP y testing
a3fb68f	2026-08-16	& Parte de Mio - Transcripción
0123370	2026-08-16	& feat(solver): historial del paso a paso para resolución de ejercicios
635ff54	2026-08-17	& feat(validator): módulo de sanitización y pruebas exhaustivas
80e1e2e	2026-08-17	& feat(solver): exportar tipos, parser y funciones desde index.ts

Hitos: motor lógico completo con 5 operaciones, parser implementado, 14 pruebas unitarias pasando.

34.6. Semana 5 - Interfaz, CI/CD y refinamiento (19-30 agosto)

La quinta semana concentró el desarrollo de la interfaz Vue 3, la configuración del pipeline CI/CD y el refinamiento final con los seis cambios solicitados por el usuario. Esta fue la semana de mayor valor entregado al usuario final.

Hash	Fecha	Descripción
1c7a6c8	2026-08-19	& Merge pull request #6 from aromEspinoza/sprint-2-hijos-de-linus
6308d04	2026-08-21	& docs: propuesta oficial del Equipo Linus Grupo 4
0fcb418	2026-08-21	& Merge pull request #7 equipo-4-linus-propuesta
d423096	2026-08-21	& fix: corregir nombres de equipos y crear documento de estado
c71fb66	2026-08-21	& feat: infraestructura completa para sprints 2-4
5175181	2026-08-21	& refactor: reorganiza TAREAS - cada entregable = carpeta + guía
5e8c767	2026-08-22	& feat: pruebas y documentación de uso para deploy final
74bea2f	2026-08-30	& feat(linus): mejoras en interfaz, corrección SVG, sub-selector potencia, enrutado

Hitos: interfaz completa con SVG, CI/CD operativo, 6 mejoras del Sprint 4 implementadas, despliegue en producción.

34.7. Estadísticas del proyecto

Tabla 13: Estadísticas consolidadas del proyecto

Estadística	Valor
Duración total del proyecto & 31 días (31 jul - 30 ago 2026)	
Total de commits & 57 commits	
Total de pull requests & 7 PR mergeadas	
Integrantes activos & 6 miembros	
Líneas de código TypeScript & ~200 líneas (motor + interfaz)	
Líneas de código Vue template & ~150 líneas	
Pruebas unitarias & 30 casos (14 del motor, 16 integración)	
Cobertura de código & 98 %	
Archivos de documentación (.md) & 8 archivos	
Sprints completados & 4 sprints	
Bugs corregidos en Sprint 4 & 2 bugs críticos + 4 mejoras	
Tiempo de compilación de producción & < 10 segundos	

% =====
 SECCIÓN 11: RESULTADOS Y DISCUSIÓN % =====

35. Resultados y Discusión

35.1. Funcionalidades implementadas vs planificadas

35.2. Rendimiento medido

35.3. Comparación con el objetivo inicial

El proyecto logró cubrir el 100 % de los objetivos planificados originalmente y adicionalmente incorporó seis mejoras no planificadas surgidas del proceso iterativo de feedback con el usuario. La decisión de usar Vue 3 + Vite + TypeScript demostró ser acertada: el tiempo de desarrollo fue corto, el código es mantenible y las pruebas son confiables.

El único objetivo que quedó fuera de alcance fue el soporte para tres o más conjuntos simultáneos, lo cual se identificó desde el inicio como “trabajo futuro” y no fue comprometido para esta entrega. El diagrama SVG de dos círculos es suficiente para las operaciones básicas del curso de Lógica Simbólica.

35.4. Limitaciones conocidas y trabajo futuro

Limitación: Límite de la función potencia. La función `potencia()` es exponencial $O(2^n)$. Para $n > 20$ elementos, el tiempo de cómputo supera 2 segundos y puede bloquear el hilo principal del navegador. Solución propuesta: usar Web Workers para ejecutar la función en un hilo separado sin bloquear la UI.

Limitación: Solo dos conjuntos. La interfaz actual solo soporta dos conjuntos (A y B). Extensiones a tres conjuntos requerirán un diagrama de Venn de tres círculos y una refactorización del motor lógico para operar sobre n conjuntos de forma genérica.

Limitación: Elementos como strings. Los elementos se tratan siempre como cadenas de texto. No hay soporte para elementos con tipos especiales (números reales, vectores, intervalos). Esto limita el uso para matemáticas avanzadas.

Limitación: Sin persistencia de datos. La aplicación no guarda el estado entre sesiones. Si el usuario cierra el navegador, pierde los conjuntos que había ingresado. Solución: usar `localStorage` para persistir el estado de la sesión.

35.5. Trabajo futuro recomendado

- Soporte para tres o más conjuntos con diagrama de Venn de tres círculos (requiere SVG complejo).
- Modo paso a paso: mostrar la construcción del resultado elemento por elemento con animaciones.
- Exportación del diagrama SVG como imagen PNG o PDF de alta resolución.
- Publicación del motor lógico como paquete npm independiente (@equipo-linus/set-operations).
- Modo evaluación: el sistema propone ejercicios y evalúa las respuestas del usuario.
- Soporte para conjuntos con elementos numéricos y operaciones de cardinalidad avanzadas.
- Integración con un LMS (Moodle, Canvas) como actividad embebida vía LTI.
- Historial de operaciones: permite ver y repetir las últimas N operaciones realizadas.
- Tema oscuro (dark mode) para uso en ambientes de poca luz.

Tabla 14: Evaluación de funcionalidades implementadas vs. planificadas

Funcionalidad	Estado original	Estado final	Sprint	Resultado
Unión $A \cup B$ & Planificado & Implementado & Sprint 2 & COMPLETO				
Intersección $A \cap B$ & Planificado & Implementado & Sprint 2 & COMPLETO				
Diferencia $A \setminus B$ y $B \setminus A$ & Planificado & Implementado & Sprint 2 & COMPLETO				
Complemento A' y B' & Planificado & Implementado & Sprint 4 & COMPLETO				
Conjunto potencia $\mathcal{P}(A)$ & Planificado & Implementado & Sprint 2 & COMPLETO				
Diagrama SVG dinámico & Planificado & Implementado & Sprint 3 & COMPLETO				
Sub-selector de potencia & Solicitado Sprint 4 & Implementado & Sprint 4 & COMPLETO				
Cuadro estadístico 2^n & Solicitado Sprint 4 & Implementado & Sprint 4 & COMPLETO				
Corrección visual SVG & Solicitado Sprint 4 & Implementado & Sprint 4 & COMPLETO				
Despliegue en producción & Planificado & Implementado & Sprint 3 & COMPLETO				
CI/CD GitHub Actions & Planificado & Implementado & Sprint 3 & COMPLETO				
Pruebas unitarias ($\geq 90\%$) & Planificado & Implementado & Sprint 2 & COMPLETO				
Soporte 3+ conjuntos & NO planificado & No implementado & N/A & TRABAJO FUTURO				
Backend con base de datos & NO planificado & No implementado & N/A & FUERA DE ALCANCE				

Tabla 15: Métricas de rendimiento medidas en entorno de pruebas

Métrica	Valor me- dido	Evaluación
Arranque servidor de desarrollo (<code>npm run dev</code>)	< 500ms	Excelente
HMR (Hot Module Replacement) al guardar archivo	< 50ms	Excelente
Compilación de producción (<code>npm run build</code>)	< 10s	Bueno
Cálculo de unión/intersección/diferencia ($n < 1000$)	< 1ms	Excelente
Cálculo de complemento ($ U < 1000$)	< 1ms	Excelente
Cálculo de potencia ($n = 10$)	~2ms	Bueno
Cálculo de potencia ($n = 15$)	~50ms	Aceptable
Cálculo de potencia ($n = 20$)	~2s	Límite máximo
Tamaño del bundle de producción (JS total)	< 180KB	Bueno
Tamaño del bundle gzip	< 60KB	Excelente
Tiempo de carga inicial en navegador	< 1s	Excelente
Ejecución suite completo de pruebas	< 200ms	Excelente

- Modo de accesibilidad: descripciones de audio del diagrama SVG para usuarios con discapacidad visual.

% =====
 SECCIÓN 12: CONCLUSIONES % =====

36. Conclusiones

36.1. Logro del objetivo principal

El Equipo Linus logró desarrollar satisfactoriamente una aplicación web interactiva que cumple con todos los objetivos planificados. La herramienta permite a los estudiantes explorar visualmente las cinco operaciones fundamentales de teoría de conjuntos mediante diagramas de Venn dinámicos, con una interfaz intuitiva, código abierto y en idioma español.

36.2. Calidad del motor lógico

La implementación del motor de operaciones en TypeScript, utilizando la API nativa Set de ES6, resultó elegante, eficiente y matemáticamente correcta. Las 30 pruebas unitarias con cobertura del 98 % garantizan la correctitud de todas las operaciones. El diseño funcional puro (sin efectos secundarios) facilita el testing y la composición de funciones.

36.3. Efectividad de Vue 3 para aplicaciones educativas

Vue 3 con la Composition API demostró ser la elección correcta para este tipo de aplicación interactiva. La reactividad automática (`ref`, `computed`) eliminó la necesidad de código imperativo para actualizar la UI, resultando en un código más limpio y mantenible. El ciclo de desarrollo con Vite fue notablemente rápido.

36.4. Valor del desarrollo ágil iterativo

Los cuatro sprints de desarrollo permitieron incorporar feedback real del usuario en cada iteración. Las seis mejoras del Sprint 4 - surgidas de una revisión con el producto funcionando - mejoraron significativamente tanto la correctitud matemática (corrección del complemento y del SVG) como la experiencia de usuario (cuadro estadístico, sub-selector, textos normalizados). Esto demuestra el valor del ciclo de feedback rápido en el desarrollo ágil.

36.5. Importancia de la documentación continua

La elaboración de este informe técnico, del manual de usuario y de la documentación del código en paralelo al desarrollo facilitó la comunicación dentro del equipo y con el docente. La documentación “post-hoc” (escrita al final) tiende a ser incompleta y menos precisa que la documentación desarrollada durante el proceso.

36.6. Aprendizajes del equipo

El proyecto permitió al equipo desarrollar competencias técnicas en Vue 3, TypeScript, Git Flow, pruebas unitarias y CI/CD. Igualmente importante fue el desarrollo de competencias colaborativas: gestión de conflictos de merge, revisión de código, comunicación de estado del proyecto y gestión del tiempo en sprints. Estas competencias son directamente transferibles al entorno profesional.

36.7. Impacto educativo potencial

La herramienta tiene potencial para impactar positivamente el aprendizaje de la teoría de conjuntos en cursos de lógica, matemática discreta e informática. Al ser de código abierto y estar disponible públicamente, puede ser adoptada por otros equipos docentes más allá de la UNPRG.

La próxima iteración debería incluir un estudio de usabilidad con estudiantes reales para medir el impacto educativo con datos empíricos.

% =====
 REFERENCIAS BIBLIOGRÁFICAS % =====

Referencias Bibliográficas

- [1] P. R. Halmos, *Naive Set Theory*. Van Nostrand Reinhold, 1960. ISBN 978-0-387-90092-6.
- [2] G. Cantor, “Beiträge zur Begründung der transfiniten Mengenlehre,” *Mathematische Annalen*, vol. 46, no. 4, pp. 481–512, 1895. <https://doi.org/10.1007/BF02124929>
- [3] J. Venn, “On the diagrammatic and mechanical representation of propositions and reasonings,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 9, no. 59, pp. 1–18, 1880.
- [4] Vue.js Team, “Vue 3 Documentation - Composition API,” 2024. <https://vuejs.org/guide/extras/composition-api-faq>
- [5] Microsoft, “TypeScript Handbook - Generics,” 2024. <https://www.typescriptlang.org/docs/handbook/2/generics.html>
- [6] ECMA International, “ECMAScript 2023 Language Specification - Set Objects (Sección 23.2),” 2023. <https://www.ecma-international.org/ecma-262/>
- [7] Vite Team, “Vite Documentation - Why Vite?,” 2024. <https://vitejs.dev/guide/why.html>
- [8] Vitest Team, “Vitest Documentation - Getting Started,” 2024. <https://vitest.dev/guide/>
- [9] SciELO, “Criterios, política y procedimientos para la admisión y la permanencia de periódicos científicos en la Red SciELO,” 2023. <https://www.scielo.cl/about/criterios-scielo/>
- [10] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed. McGraw-Hill Education, 2015. ISBN 978-0-07-802212-8.
- [11] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. ISBN 978-0-13-235088-4.
- [12] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997. ISBN 978-0-201-89684-1.
- [13] GitHub, “GitHub Actions Documentation,” 2024. <https://docs.github.com/es/actions>
- [14] W3C, “Scalable Vector Graphics (SVG) 2,” 2018. <https://www.w3.org/TR/SVG2/>
- [15] J. Sutherland, *Scrum: The Art of Doing Twice the Work in Half the Time*. Crown Business, 2014. ISBN 978-0-385-34645-0.

```

%=====
APÉNDICES %=====
% — APÉNDICE A —

```

A. Apéndice A - Código Completo del Motor Lógico

A continuación se presenta el código fuente completo y comentado del archivo `src/lib/sets/operations.ts` en su versión final (Sprint 4, commit 74bea2f).

Listing 18: `src/lib/sets/operations.ts` - versión completa comentada

```

1  /**
2   * @file operations.ts
3   * @description Motor logico de operaciones de teoria de conjuntos.
4   *             Implementa union, interseccion, diferencia, complemento
5   *             y conjunto potencia sobre objetos Set<T> nativos de ES6.
6   *
7   * @author Equipo Linus - Grupo 4
8   * @course Logica Simbolica - UNPRG 2026
9   * @license MIT
10  *
11  * DISEÑO: funciones puras sin efectos secundarios.
12  * - Cada funcion retorna un nuevo Set sin modificar los argumentos.
13  * - No hay dependencias externas (solo JavaScript estandar).
14  * - Las funciones son genericas (Set<T>) para maxima reutilizacion.
15  */
16
17 /**
18  * union - Calcula la union matematica A U B.
19  *
20  * @definition A U B = { x | x in A OR x in B }
21  * @param a Primer conjunto.
22  * @param b Segundo conjunto.
23  * @returns Un nuevo Set con todos los elementos de A y B.
24  * @complexity O(|A| + |B|) tiempo, O(|A| + |B|) espacio.
25  *
26  * @example
27  * union(new Set(['a', 'b']), new Set(['b', 'c'])) // Set {'a', 'b', 'c'}
28  */
29 export function union<T>(a: Set<T>, b: Set<T>): Set<T> {
30   return new Set([...a, ...b]);
31 }
32
33 /**
34  * interseccion - Calcula la interseccion matematica A n B.
35  *
36  * @definition A n B = { x | x in A AND x in B }
37  * @param a Primer conjunto.
38  * @param b Segundo conjunto.
39  * @returns Un nuevo Set con los elementos comunes a A y B.
40  * @complexity O(|A|) tiempo, O(min(|A|, |B|)) espacio.
41  *
42  * @example
43  * interseccion(new Set(['a', 'b', 'c']), new Set(['b', 'c'])) // Set {'b', 'c'}
44  */
45 export function interseccion<T>(a: Set<T>, b: Set<T>): Set<T> {
46   return new Set([...a].filter(x => b.has(x)));
47 }
48
49 /**
50  * diferencia - Calcula la diferencia A \ B.
51  *

```

```

52 * @definition  $A \setminus B = \{ x \mid x \text{ in } A \text{ AND } x \text{ not-in } B \}$ 
53 * @param a Conjunto del que se restan elementos.
54 * @param b Conjunto cuyos elementos se excluyen.
55 * @returns Un nuevo Set con los elementos de A que no estan en B.
56 * @complexity  $O(|A|)$  tiempo,  $O(|A|)$  espacio.
57 *
58 * @example
59 * diferencia(new Set(['a','b','c']), new Set(['b'])) // Set {'a','c'}
60 */
61 export function diferencia<T>(a: Set<T>, b: Set<T>): Set<T> {
62     return new Set([...a].filter(x => !b.has(x)));
63 }
64
65 /**
66 * complemento - Calcula el complemento de A respecto al universo U.
67 *
68 * @definition  $A' = U \setminus A = \{ x \mid x \text{ in } U \text{ AND } x \text{ not-in } A \}$ 
69 * @param universo El conjunto universal del problema.
70 * @param a El conjunto cuyo complemento se calcula.
71 * @returns Un nuevo Set con los elementos de U que no pertenecen a A.
72 * @complexity  $O(|U|)$  tiempo,  $O(|U|)$  espacio.
73 *
74 * @example
75 * complemento(new Set(['a','b','c']), new Set(['a'])) // Set {'b','c'}
76 */
77 export function complemento<T>(universo: Set<T>, a: Set<T>): Set<T> {
78     return diferencia(universo, a); // Reutiliza diferencia() - DRY principle
79 }
80
81 /**
82 * potencia - Calcula el conjunto potencia  $P(A)$ .
83 *
84 * @definition  $P(A) = \{ S \mid S \subseteq A \}$ 
85 *  $|P(A)| = 2^{|A|}$ 
86 *
87 * ALGORITMO: mascara de bits (bit-mask enumeration).
88 * Para cada entero i de 0 a  $2^n-1$ , el valor binario de i
89 * determina que elementos de A incluir en el subconjunto i-esimo.
90 * Si el bit j de i es 1, el elemento j-esimo de A se incluye.
91 *
92 * @param a El conjunto del que se calcula la potencia.
93 * @returns Un Set<Set<T>> con todos los subconjuntos de A.
94 * @complexity  $O(2^n * n)$  tiempo y espacio, donde  $n = |A|$ .
95 * @warning EXPONENCIAL. Usar solo para  $|A| \leq 20$ .
96 *
97 * @example
98 * potencia(new Set(['a','b'])) // Set { {}, {'a'}, {'b'}, {'a','b'} }
99 */
100 export function potencia<T>(a: Set<T>): Set<Set<T>> {
101     const elementos = Array.from(a); // Necesario para acceso por indice
102     const subconjuntos = new Set<Set<T>>();
103     const numSubconjuntos = Math.pow(2, elementos.length); //  $2^n$ 
104
105     for (let i = 0; i < numSubconjuntos; i++) {
106         const subconjunto = new Set<T>();
107         for (let j = 0; j < elementos.length; j++) {
108             // Comprueba si el bit j del entero i esta activado
109             // (1 << j) crea una mascara con solo el bit j en 1
110             // (i & mascara) es distinto de 0 si el bit j de i es 1
111             if ((i & (1 << j)) !== 0) {
112                 subconjunto.add(elementos[j]);
113             }
114         }
115     }
116 }

```

```
115     subconjuntos.add(subconjunto);  
116 }  
117 return subconjuntos;  
118 }
```

% — APÉNDICE B —

B. Apéndice B - Código de Pruebas Unitarias

A continuación se presenta el código completo del archivo de pruebas unitarias `src/lib/sets/operations.test.ts`

Listing 19: `src/lib/sets/operations.test.ts` - pruebas completas

```

1  /**
2   * @file operations.test.ts
3   * @description Pruebas unitarias del motor logico de conjuntos.
4   * @framework Vitest (compatible con Vite)
5   * @coverage 98% de lineas, 95% de ramas
6   */
7
8  import { describe, it, expect } from 'vitest';
9  import {
10    union, interseccion, diferencia, complemento, potencia
11  } from './operations';
12
13  // =====
14  // SUITE: union
15  // =====
16  describe('union()', () => {
17    it('une conjuntos sin elementos comunes', () => {
18      expect(union(new Set(['a','b']), new Set(['c','d'])))
19        .toEqual(new Set(['a','b','c','d']));
20    });
21
22    it('une conjuntos con elementos comunes', () => {
23      expect(union(new Set(['a','b']), new Set(['b','c'])))
24        .toEqual(new Set(['a','b','c']));
25    });
26
27    it('union con A vacio', () => {
28      expect(union(new Set(), new Set(['a','b'])))
29        .toEqual(new Set(['a','b']));
30    });
31
32    it('union con B vacio', () => {
33      expect(union(new Set(['a','b']), new Set()))
34        .toEqual(new Set(['a','b']));
35    });
36
37    it('union de dos vacios', () => {
38      expect(union(new Set(), new Set())).toEqual(new Set());
39    });
40
41    it('union idempotente (A con A)', () => {
42      expect(union(new Set(['a','b']), new Set(['a','b'])))
43        .toEqual(new Set(['a','b']));
44    });
45  });
46
47  // =====
48  // SUITE: interseccion
49  // =====
50  describe('interseccion()', () => {
51    it('interseccion parcial', () => {
52      expect(interseccion(new Set(['a','b','c']), new Set(['b','c','d'])))
53        .toEqual(new Set(['b','c']));
54    });
55  });

```

```

56   it('conjuntos disjuntos -> vacio', () => {
57       expect(interseccion(new Set(['a','b']), new Set(['c','d'])))
58       .toEqual(new Set());
59   });
60
61   it('A vacio -> resultado vacio', () => {
62       expect(interseccion(new Set(), new Set(['a','b'])))
63       .toEqual(new Set());
64   });
65
66   it('conjuntos identicos -> mismo conjunto', () => {
67       expect(interseccion(new Set(['a','b']), new Set(['a','b'])))
68       .toEqual(new Set(['a','b']));
69   });
70
71   it('interseccion unitaria', () => {
72       expect(interseccion(new Set(['a','b','c']), new Set(['a'])))
73       .toEqual(new Set(['a']));
74   });
75 });
76
77 // =====
78 // SUITE: diferencia
79 // =====
80 describe('diferencia()', () => {
81     it('diferencia parcial A-B', () => {
82         expect(diferencia(new Set(['a','b','c']), new Set(['b','c'])))
83         .toEqual(new Set(['a']));
84     });
85
86     it('B no tiene comunes con A -> retorna A', () => {
87         expect(diferencia(new Set(['a','b']), new Set(['c','d'])))
88         .toEqual(new Set(['a','b']));
89     });
90
91     it('A subconjunto de B -> vacio', () => {
92         expect(diferencia(new Set(['a']), new Set(['a','b','c'])))
93         .toEqual(new Set());
94     });
95
96     it('A vacio -> vacio', () => {
97         expect(diferencia(new Set(), new Set(['a','b'])))
98         .toEqual(new Set());
99     });
100
101     it('B vacio -> retorna A', () => {
102         expect(diferencia(new Set(['a','b']), new Set()))
103         .toEqual(new Set(['a','b']));
104     });
105
106     it('A igual B -> vacio', () => {
107         expect(diferencia(new Set(['a','b']), new Set(['a','b'])))
108         .toEqual(new Set());
109     });
110 });
111
112 // =====
113 // SUITE: complemento
114 // =====
115 describe('complemento()', () => {
116     it('complemento parcial', () => {
117         const U = new Set(['a','b','c','d']);
118         expect(complemento(U, new Set(['a','b'])))

```

```

119     .toEqual(new Set(['c','d']));
120   });
121
122   it('complemento del vacio = U', () => {
123     const U = new Set(['a','b']);
124     expect(complemento(U, new Set())).toEqual(U);
125   });
126
127   it('complemento de U = vacio', () => {
128     const U = new Set(['a','b']);
129     expect(complemento(U, U)).toEqual(new Set());
130   });
131 });
132
133 // =====
134 // SUITE: potencia
135 // =====
136 describe('potencia()', () => {
137   it('potencia del vacio: 1 subconjunto (el vacio)', () => {
138     expect(potencia(new Set()).size).toBe(1);
139   });
140
141   it('potencia({a}): 2 subconjuntos', () => {
142     expect(potencia(new Set(['a'])).size).toBe(2);
143   });
144
145   it('potencia({a,b}): 4 subconjuntos', () => {
146     expect(potencia(new Set(['a','b'])).size).toBe(4);
147   });
148
149   it('potencia({a,b,c}): 8 subconjuntos', () => {
150     expect(potencia(new Set(['a','b','c'])).size).toBe(8);
151   });
152
153   it('potencia({a,b,c,d}): 16 subconjuntos', () => {
154     expect(potencia(new Set(['a','b','c','d'])).size).toBe(16);
155   });
156
157   it('potencia verifica formula 2^n', () => {
158     for (let n = 0; n <= 10; n++) {
159       const elementos = Array.from({length: n}, (_, i) => `e${i}`);
160       expect(potencia(new Set(elementos)).size).toBe(Math.pow(2, n));
161     }
162   });
163 });

```

% — APÉNDICE C —

C. Apéndice C - Configuración del Proyecto

C.1. package.json

Listing 20: package.json

```
1 {
2   "name": "logica-simbolica-global",
3   "version": "1.0.0",
4   "private": true,
5   "scripts": {
6     "dev": "vite",
7     "build": "vue-tsc && vite build",
8     "preview": "vite preview",
9     "test": "vitest run",
10    "test:ui": "vitest --ui",
11    "test:cov": "vitest run --coverage"
12  },
13  "dependencies": {
14    "vue": "^3.4.0"
15  },
16  "devDependencies": {
17    "@vitejs/plugin-vue": "^5.0.0",
18    "@vue/test-utils": "^2.4.0",
19    "jsdom": "^24.0.0",
20    "typescript": "^5.3.0",
21    "vite": "^5.0.0",
22    "vitest": "^1.2.0",
23    "vue-tsc": "^2.0.0"
24  }
25 }
```

C.2. tsconfig.json

Listing 21: tsconfig.json

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "useDefineForClassFields": true,
5     "module": "ESNext",
6     "moduleResolution": "bundler",
7     "strict": true,
8     "jsx": "preserve",
9     "resolveJsonModule": true,
10    "isolatedModules": true,
11    "noEmit": true,
12    "lib": ["ES2022", "DOM", "DOM.Iterable"],
13    "baseUrl": ".",
14    "paths": {
15      "@/*": ["src/*"]
16    }
17  },
18  "include": ["src/**/*.ts", "src/**/*.vue"]
19 }
```

C.3. vite.config.ts

Listing 22: vite.config.ts

```
1 import { defineConfig } from 'vite'
2 import vue from '@vitejs/plugin-vue'
3 import path from 'path'
4
5 export default defineConfig({
6   plugins: [vue()],
7   resolve: {
8     alias: {
9       '@': path.resolve(__dirname, './src'),
10    },
11  },
12  test: {
13    environment: 'jsdom',
14    globals: true,
15    include: ['src/**/*.test.ts'],
16    coverage: {
17      provider: 'v8',
18      include: ['src/lib/**/*.ts'],
19    },
20  },
21 })
```

% — APÉNDICE D —

D. Apéndice D - Glosario de Términos

API (Application Programming Interface):

Conjunto de definiciones, protocolos y herramientas que permiten la comunicación entre diferentes componentes de software. En este proyecto, “API” refiere tanto a la API Set de ES6 como a la API pública del módulo `operations.ts`.

Bundle:

El archivo o conjunto de archivos JavaScript/CSS resultante del proceso de compilación de Vite. Contiene todo el código necesario para ejecutar la aplicación en el navegador, optimizado para minimizar el tamaño y maximizar el rendimiento.

CI/CD (Continuous Integration / Continuous Delivery):

Práctica de ingeniería de software que consiste en integrar el código frecuentemente (CI) y desplegar automáticamente cada versión aprobada (CD). En este proyecto, GitHub Actions ejecuta las pruebas y despliega automáticamente en cada push a main.

Composition API:

API de Vue 3 que permite organizar la lógica del componente por funcionalidad (usando `ref`, `computed`, `watch`, etc.) en lugar de por tipo de opción (como en Options API). Facilita la reutilización de lógica entre componentes mediante composables.

Computed (propiedad computada):

En Vue 3, una propiedad computada es una función que se recalcula automáticamente cuando cambian sus dependencias reactivas. Equivalente a una “fórmula” en hojas de cálculo que se actualiza sola al cambiar los datos de entrada.

DRY (Don’t Repeat Yourself):

Principio de desarrollo de software que establece que “cada pieza de conocimiento debe tener una representación única, inequívoca y autorizada dentro del sistema”. En `operations.ts`, `complemento()` reutiliza `diferencia()` en lugar de duplicar código.

ES6 (ECMAScript 2015):

Versión 6 del estándar ECMAScript (JavaScript). Introdujo características clave como: `let/const`, arrow functions, clases, módulos (`import/export`), template literals, destructuring y la API Set/Map.

Generic / Genérica (función genérica):

En TypeScript, una función genérica es aquella que puede operar sobre diferentes tipos de datos sin perder la seguridad de tipos. Se declaran con `<T>` y permiten que el mismo código funcione con `Set<string>`, `Set<number>`, etc.

HMR (Hot Module Replacement):

Característica de Vite que reemplaza en el navegador solo el módulo modificado al guardar un archivo, sin recargar la página completa. Permite ver los cambios instantáneamente (<50ms) durante el desarrollo.

SFC (Single File Component):

Formato de archivo `.vue` de Vue que encapsula en un único archivo la plantilla HTML (`<template>`), la lógica JavaScript/TypeScript (`<script>`) y los estilos CSS (`<style>`). Facilita la organización del código por componente.

Set (tipo de dato):

Estructura de datos de JavaScript ES6 que representa una colección de valores únicos (sin duplicados). Garantiza la unicidad automáticamente y ofrece comprobación de pertenencia $O(1)$ con el método `.has()`.

Spread operator (...):

Operador de JavaScript ES6 que “expande” un iterable (como un Set o Array) en sus elementos individuales. En `[...a, ...b]`, ambos Sets se convierten en sus elementos, que luego se concatenan en un nuevo array.

SRP (Single Responsibility Principle):

Principio de diseño de software que establece que una clase o función debe tener una sola razón para cambiar. En `operations.ts`, cada función tiene una única responsabilidad: calcular una operación específica de conjuntos.

SVG (Scalable Vector Graphics):

Formato de imagen vectorial basado en XML, nativo del navegador web. Las imágenes SVG escalan sin pérdida de calidad a cualquier resolución. En este proyecto, el diagrama de Venn se implementa como SVG inline en Vue.

Tree-shaking:

Optimización del bundler (Vite) que elimina el código no utilizado del bundle final. Las exportaciones nombradas de `operations.ts` permiten el tree-shaking: si la interfaz no usa “potencia”, esa función se excluye del bundle.

TypeScript:

Superconjunto tipado de JavaScript desarrollado por Microsoft. Agrega anotaciones de tipo opcionales que son verificadas en tiempo de compilación. El código TypeScript se transpila a JavaScript estándar para ejecutarse en el navegador.

Vitest: Framework de pruebas unitarias moderno, compatible con Vite. Reutiliza la configuración de Vite (alias, plugins) sin necesidad de configuración adicional. Es significativamente más rápido que Jest para proyectos con Vite.

E. Estructura del repositorio (Árbol Sinergia)

APÉNDICE B — ESTRUCTURA COMPLETA DEL REPOSITORIO

1

```
=====
Estructura del proyecto LogiLearn con los módulos del Equipo Sinergia: - src/lib/truth-table/ (evaluator.ts, evaluator.test.ts) - src/pages/tablas/ (index.vue) - src/pages/leyes-logicas/ - src/pages/ejercicios/ - src/pages/progreso/ - src/components/layout/ (AppNavBar.vue) - src/components/ui/ (Button.vue, Card.vue, ToggleSwitch.vue, Badge.vue)
```

F. Procedencia de los capítulos

Sin reescritura: Caps de Sinergia de `informe_sinergia.tex` (secciones 1–4, 6, 7, 9, 10 y apéndices); Hijos de `main.tex` L193–506; Modus de `.txt` Caps I, II, III, V, VI, VII; Linus de `informe_Linus.tex` salvo el Manual (L942–1150); árbol Sinergia del Apéndice B.